Setting up

```
# Data handlers
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Models
from scipy.optimize import minimize_scalar, curve_fit
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor, AdaBoostRegressor
import xgboost
from lightgbm import LGBMRegressor
from catboost import CatBoostRegressor
import tensorflow as tf


# Preprocessors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Evaluators
from sklearn.metrics import r2_score, mean_absolute_error as mae,
mean_squared_error as mse

df = pd.read_csv('cleaned data.csv')
df.head()
```

	variable 1	variable 2	Output
0	30	30	2315.7
1	30	35	1932.6
2	30	40	1592.2
3	30	45	1316.6
4	30	50	1099.5

```
X = df.drop(columns = 'Output')
Y = df['Output']
```

Plotting function

```
var1 = df['variable 1'].unique()
var2 = df['variable 2'].unique()


def plot2d_var1const(model):
```

```
    fig,ax = plt.subplots(5,5, figsize = (20,20))

    for i in range(5):
        for j in range(5):
            temp = df[df['variable 1'] == var1[5*i+j]]
            ax[i,j].plot(temp['variable 2'], temp['Output'], marker =
'o')
            ax[i,j].plot(temp['variable 2'], temp[model], marker =
'o')
            ax[i,j].set_title('variable 1 = '+str(var1[5*i+j]))

    plt.show()


def plot2d_var2const(model):

    fig,ax = plt.subplots(5,5, figsize = (20,20))

    for i in range(5):
        for j in range(5):
            temp = df[df['variable 2'] == var1[5*i+j]]
            ax[i,j].plot(temp['variable 1'], temp['Output'], marker =
'o')
            ax[i,j].plot(temp['variable 1'], temp[model], marker =
'o')
            ax[i,j].set_title('variable 2 = '+str(var2[5*i+j]))

    plt.show()

def plot3d(model):
    fig = plt.figure(figsize = (10,10))
    ax = fig.add_subplot(projection = '3d')
    ax.scatter(df['variable 1'], df['variable 2'], df['Output'],
marker = 'o', color = 'blue', alpha = 0.25)
    ax.scatter(df['variable 1'], df['variable 2'], df[model], marker =
'o', color = 'orange')
    ax.set_xlabel('Var 1')
    ax.set_ylabel('Var 2')
    ax.set_zlabel('Output')
    plt.show()
```

Custom curve fit

```
def model_eq(X,k1,k2,k3,k4,k5,k6,k7):
    y,x = X
    return k2*y*np.exp(-k1*x) + k3*np.exp(-k1*x) - k4*x*y + k5*x +
k6*y + k7
```

```
p0 = [0.1, 1.0, 1.0, 0., 0., 0., 0.]
lower_bounds = [0.001, -np.inf, -np.inf, -np.inf, -np.inf, -np.inf, -
np.inf]
upper_bounds = [np.inf, np.inf, np.inf, np.inf, np.inf, np.inf,
np.inf]
bounds = (lower_bounds, upper_bounds)

popt,pcov = curve_fit(model_eq,X.to_numpy().T,Y,p0 = p0, bounds =
bounds)

popt
```

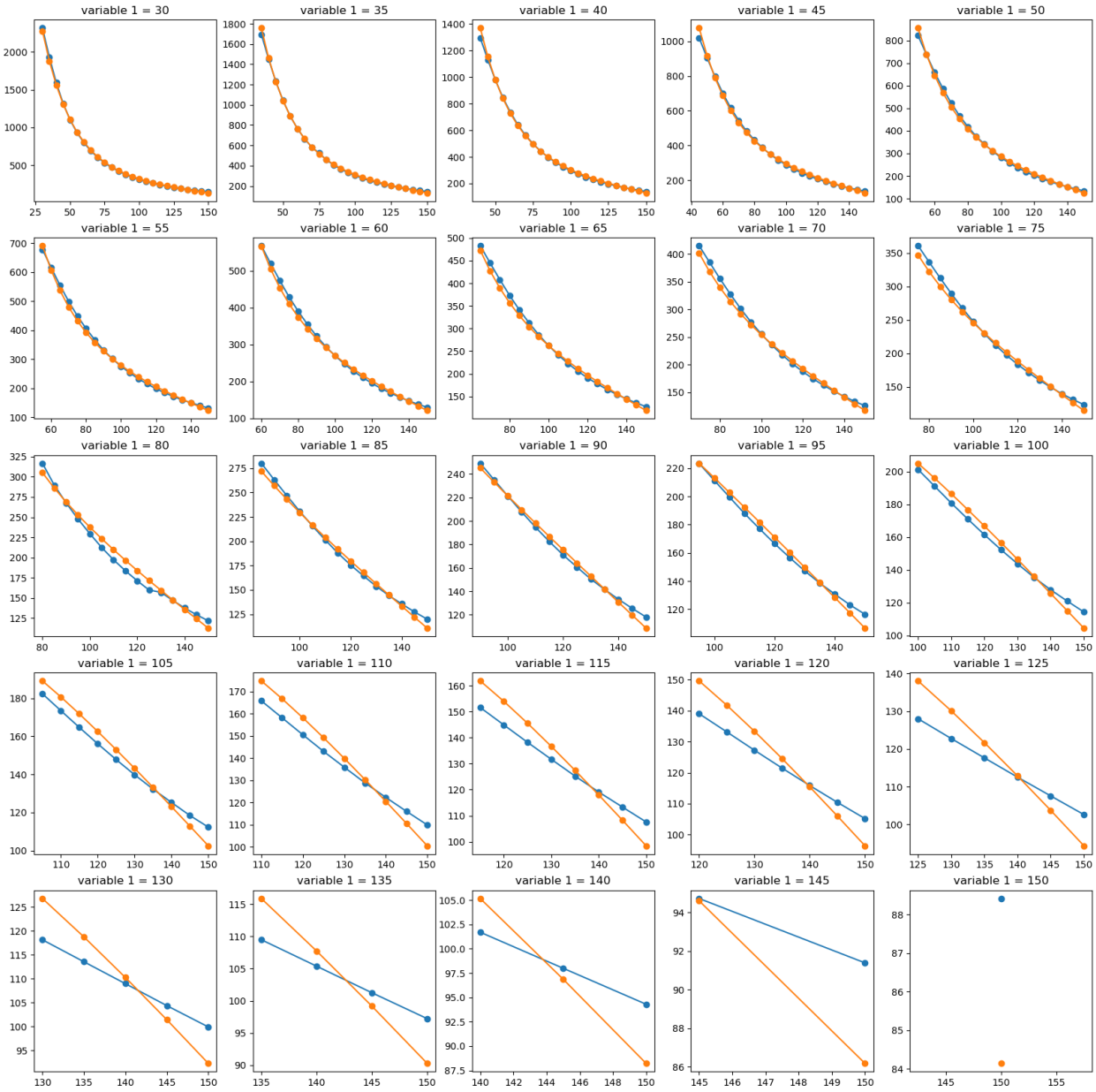
```
array([ 4.63869715e-02, -1.18760527e+02,  1.10415419e+04, -
3.69927315e-03,
       -2.52547651e+00, -8.50445143e-01,  5.13734496e+02])
```

```
df['custom'] = model_eq(X.to_numpy().T, *popt)

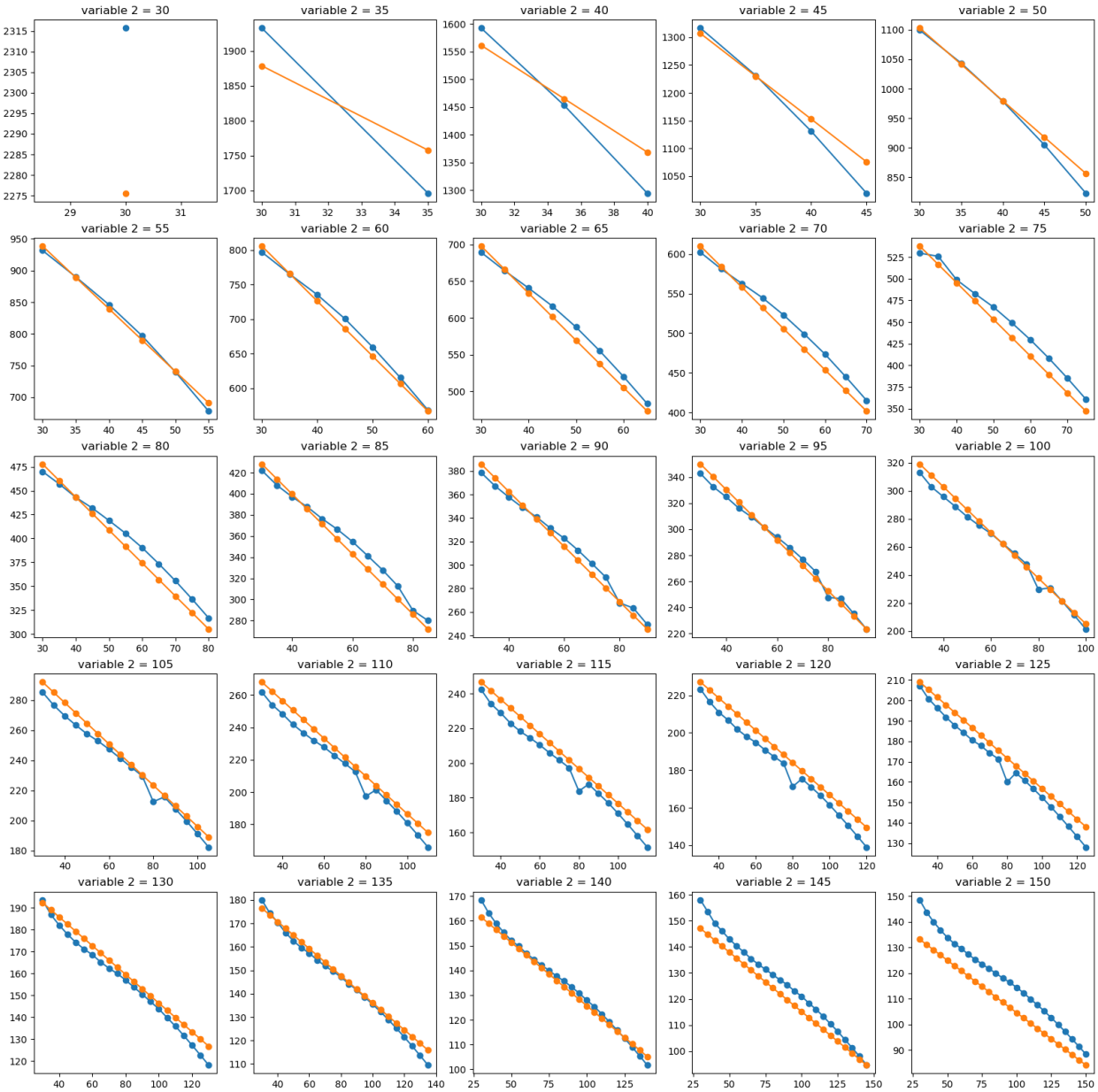
print(mse(Y, df['custom']))
```

```
113.79039603805539
```

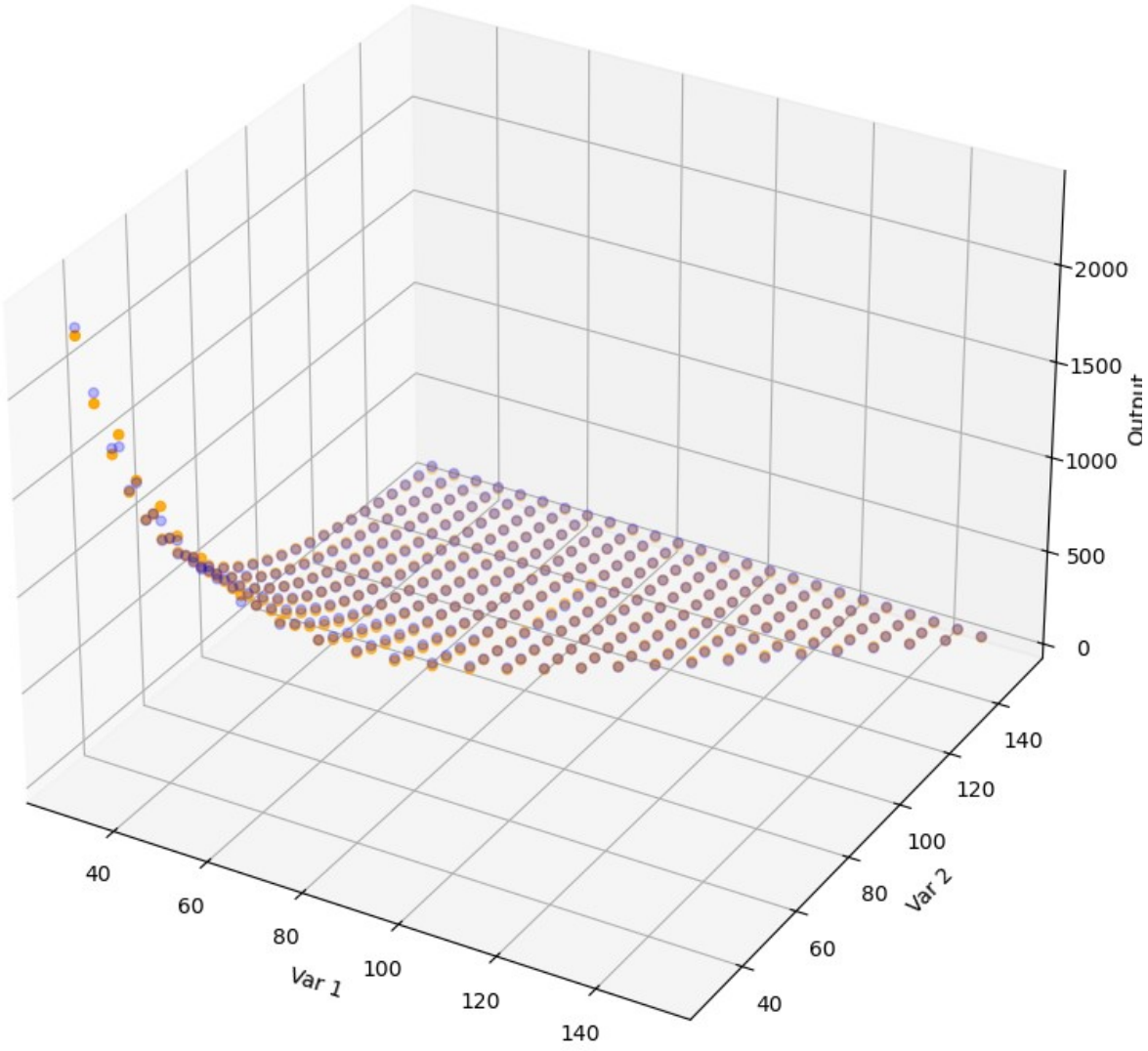
```
plot2d_var1const('custom')
```




```
plot2d_var2const('custom')
```




```
plot3d('custom')
```

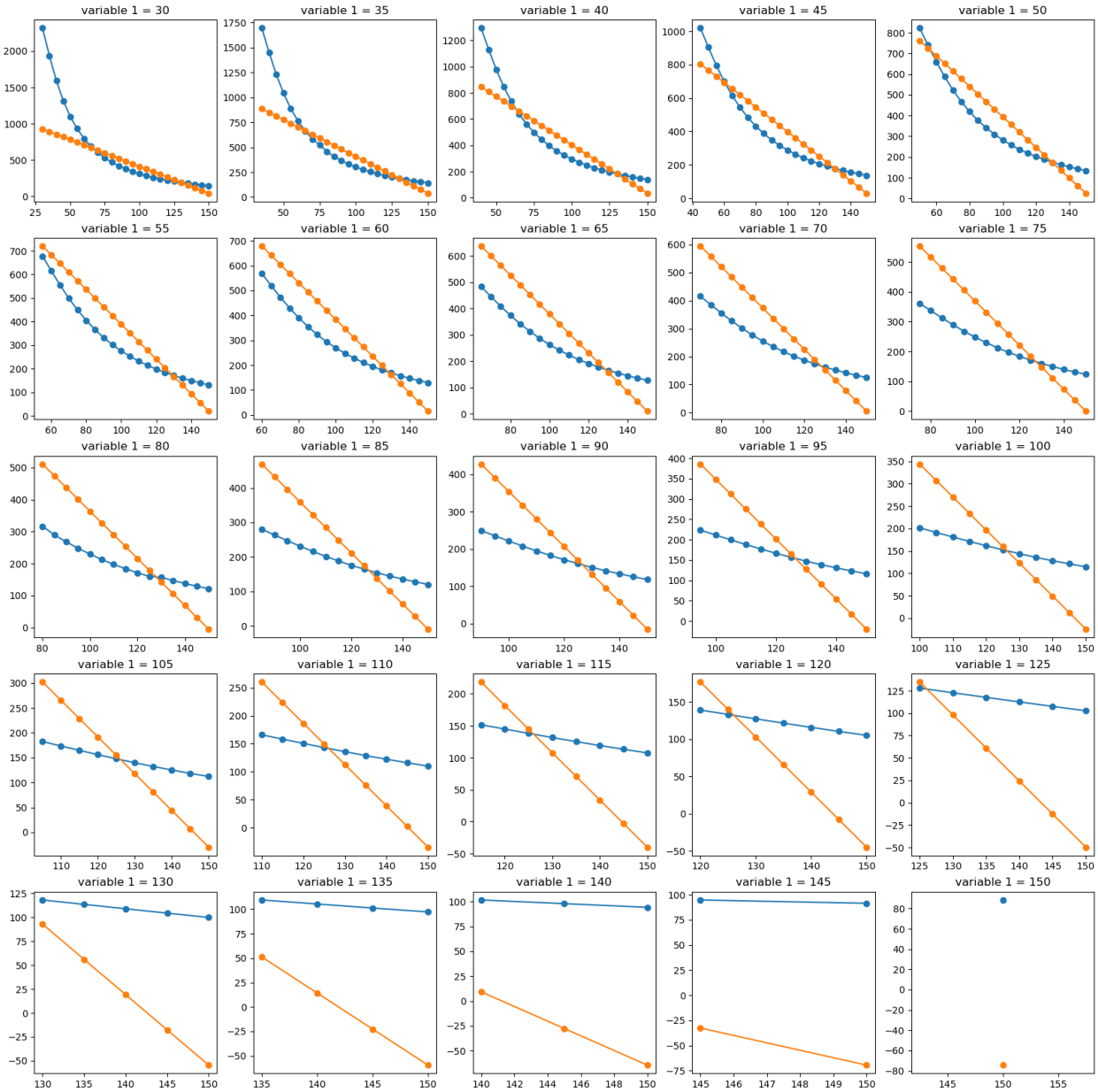


Linear Regression

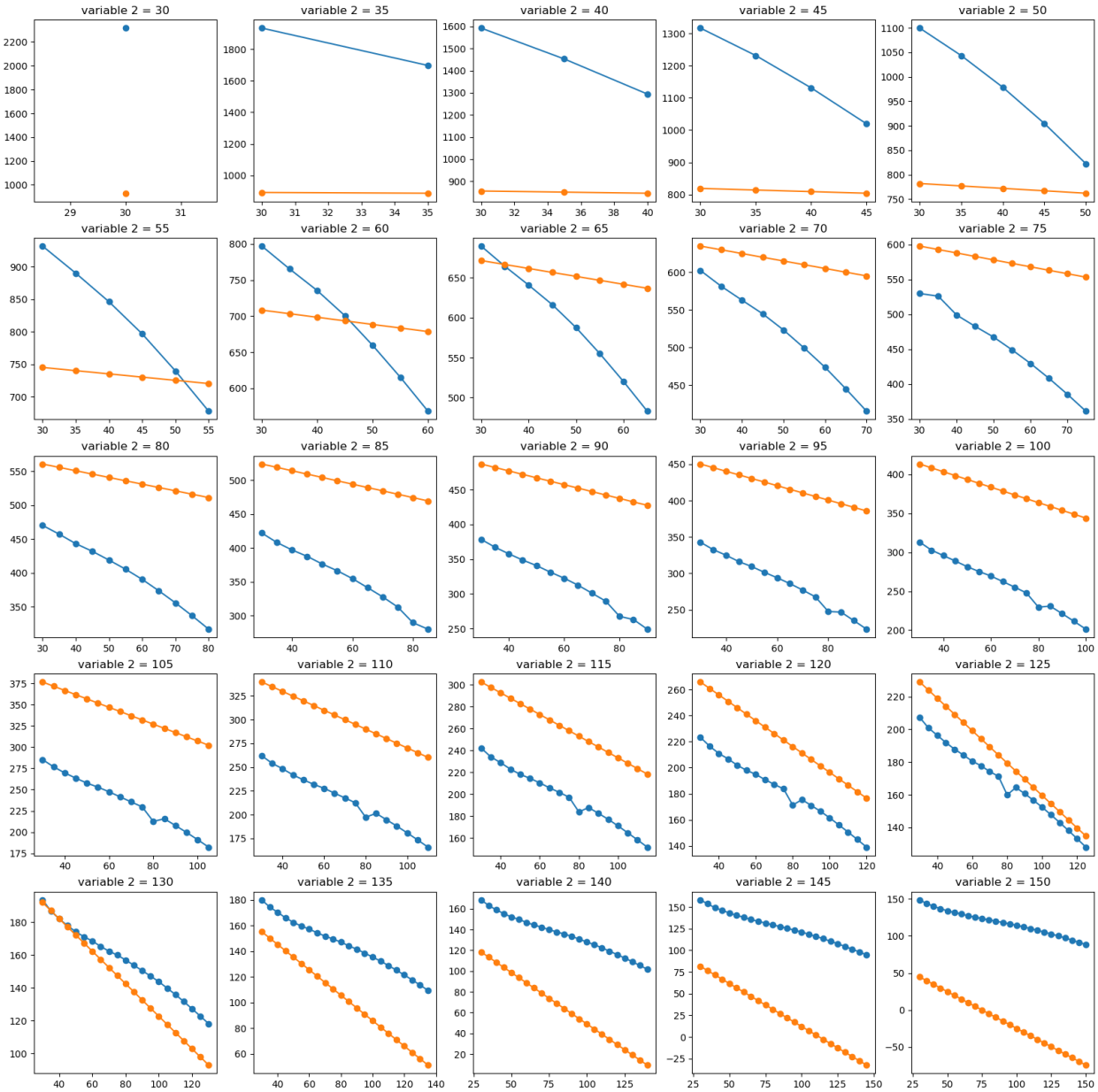
```
LinReg = LinearRegression()
LinReg.fit(X,Y)
df['Linreg'] = LinReg.predict(X)
print(mse(Y, df['Linreg']))
```

```
26034.305095744785
```

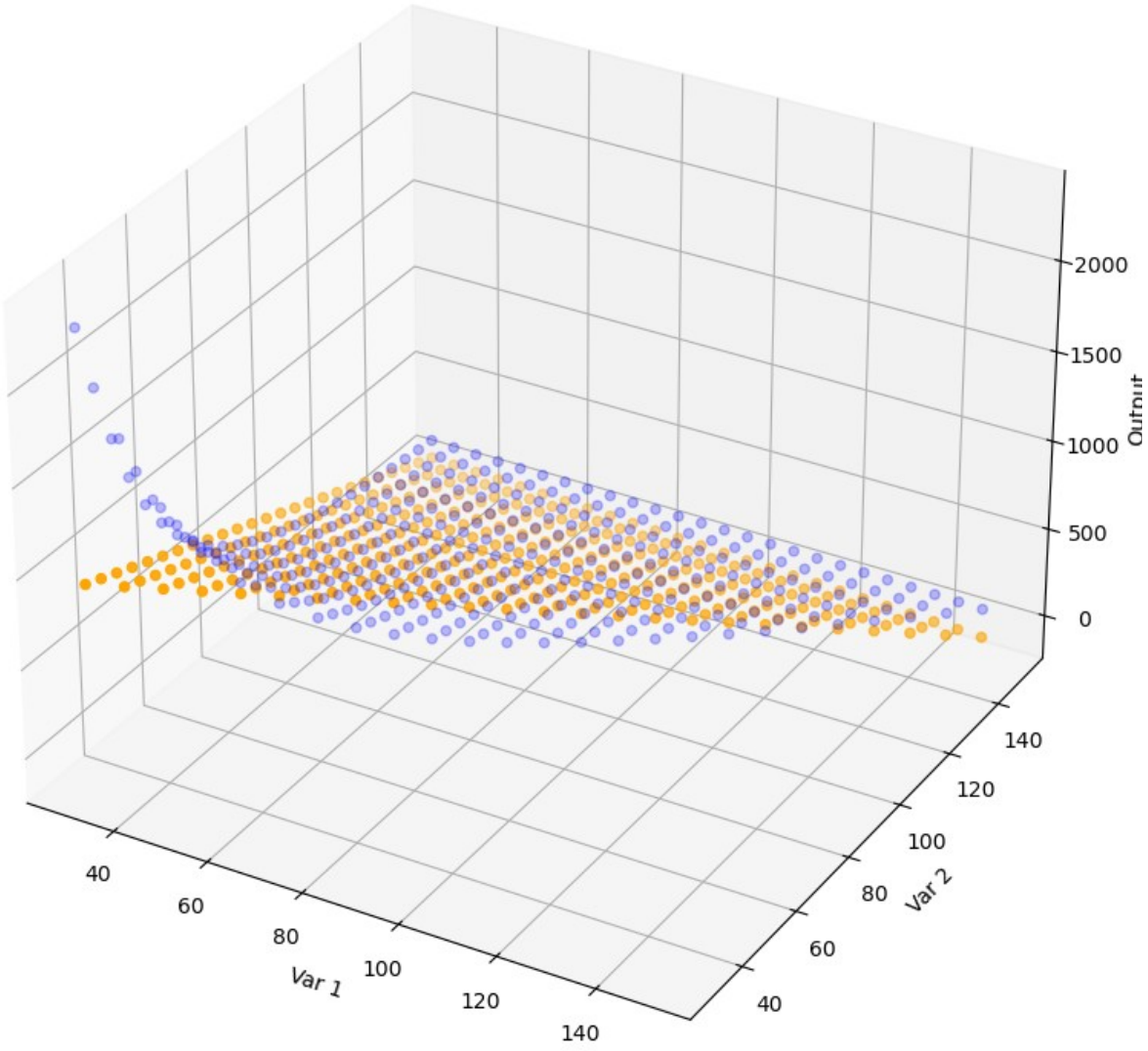
```
plot2d_var1const('Linreg')
```




```
plot2d_var2const('Linreg')
```




```
plot3d('Linreg')
```

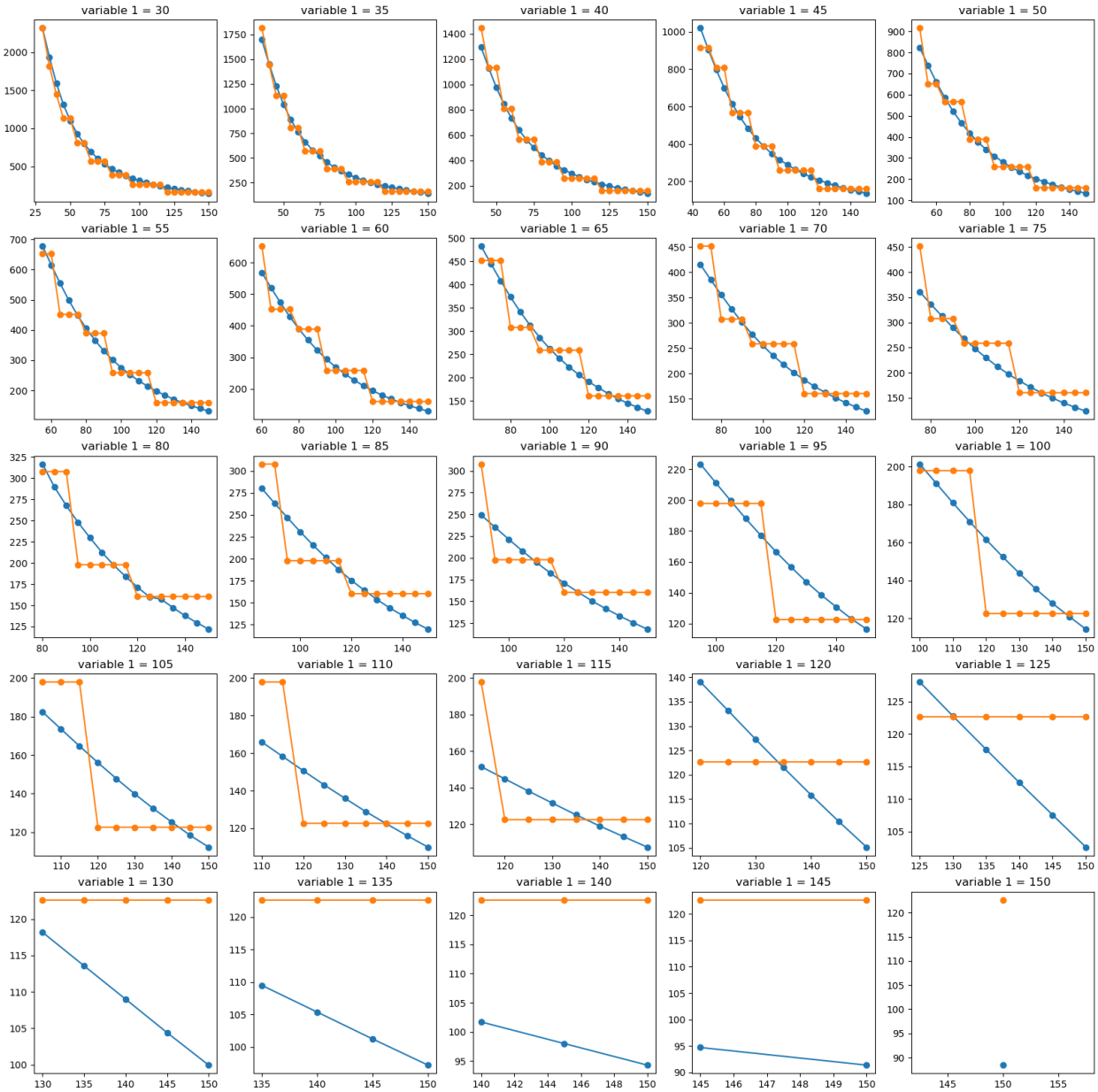


Decision Tree Regressor

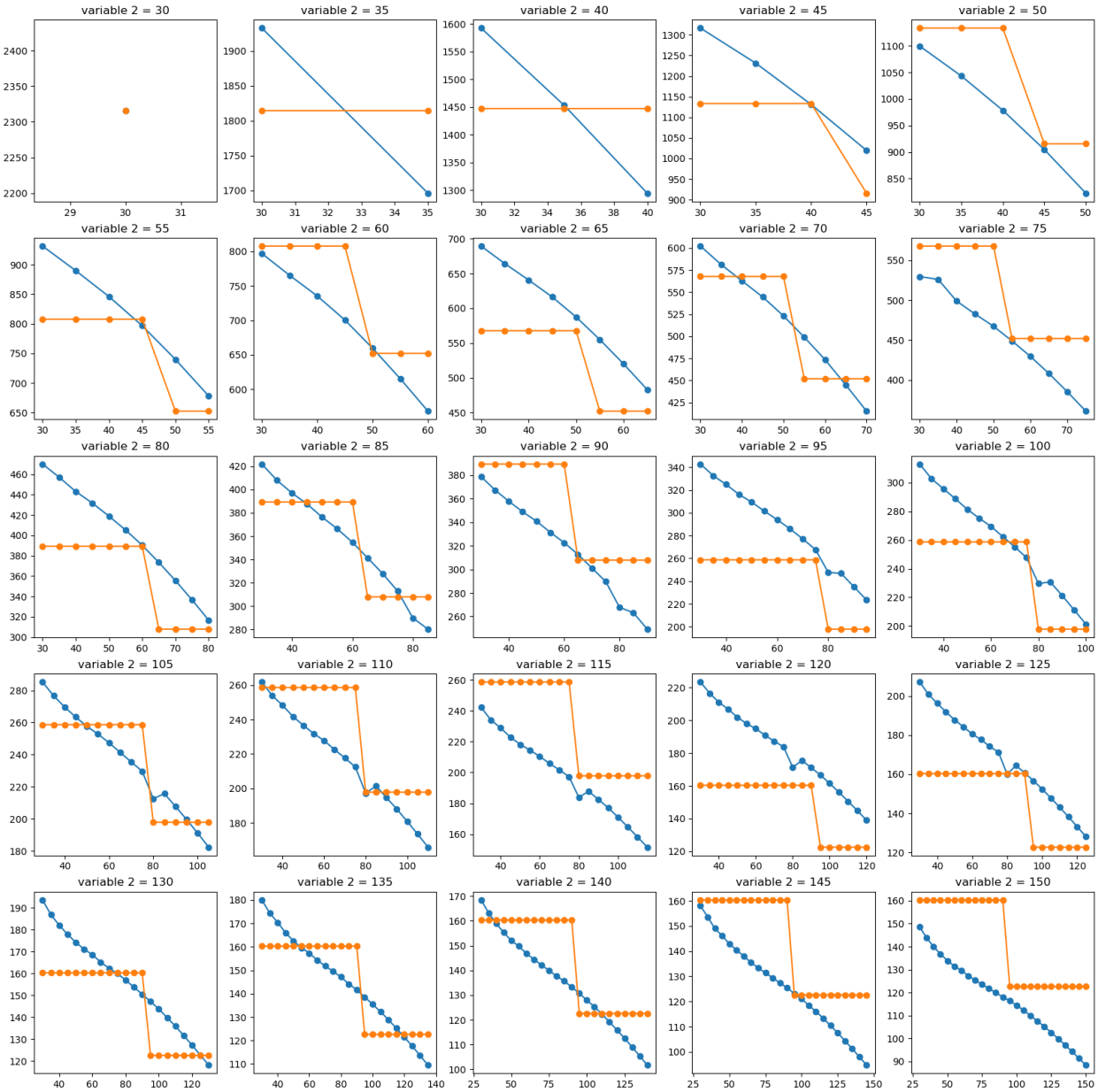
```
DTReg = DecisionTreeRegressor(random_state = 0, max_depth = 4,
min_samples_split = 2)
DTReg.fit(X,Y)
df['DTReg'] = DTReg.predict(X)
print(mse(Y, df['DTReg']))

1649.0956632380858

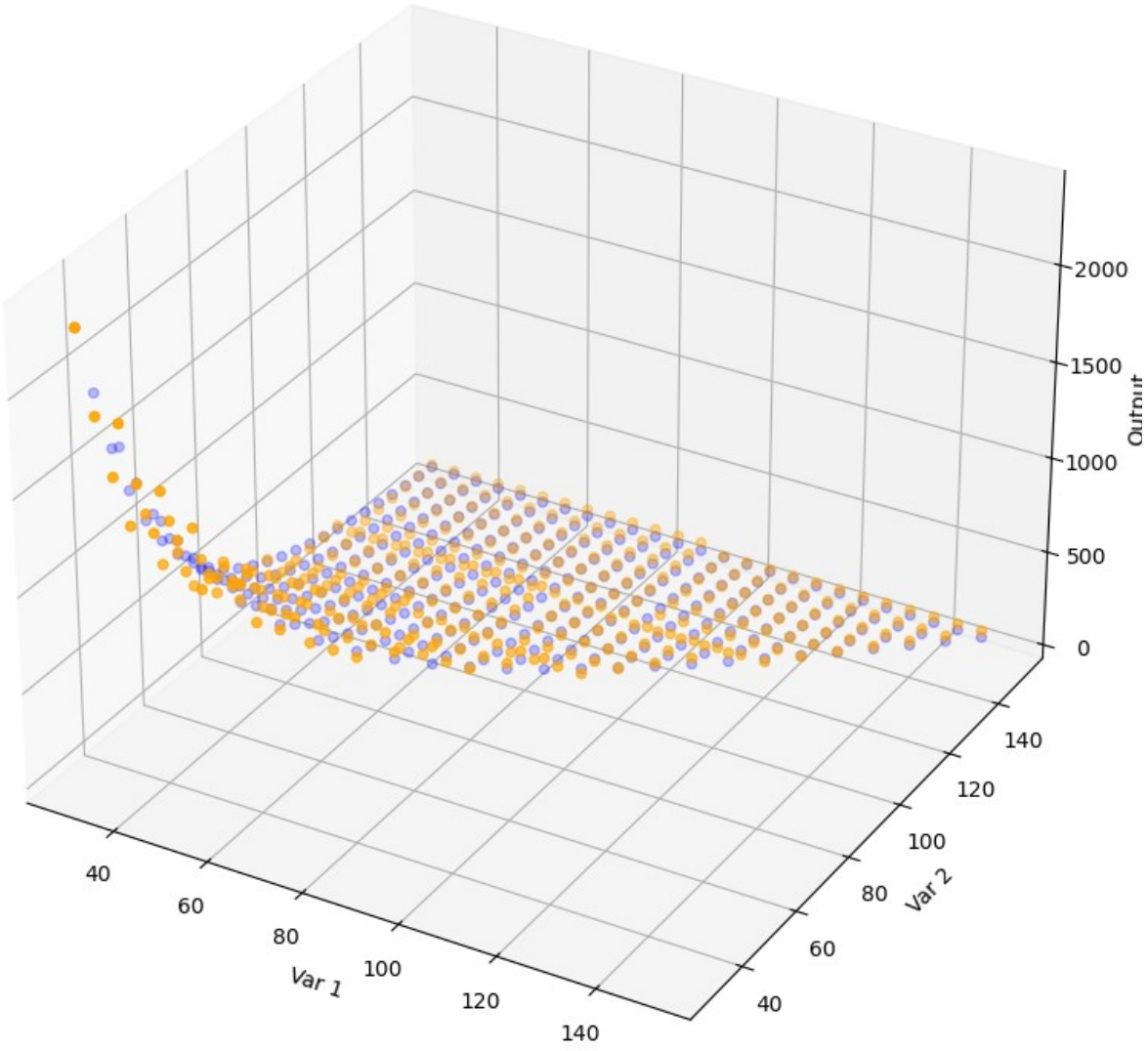
plot2d_var1const('DTReg')
```




```
plot2d_var2const('DTReg')
```




```
plot3d('DTReg')
```



Random Forest Regressor

```
RFReg = RandomForestRegressor(
    n_estimators = 10,
    random_state = 0,
    oob_score = True,
    max_depth = 8
)


RFReg.fit(X,Y)
```

```
df['RFReg'] = RFReg.predict(X)
print(mse(Y, df['RFReg']), '\n',
      RFReg.oob_score_)
```

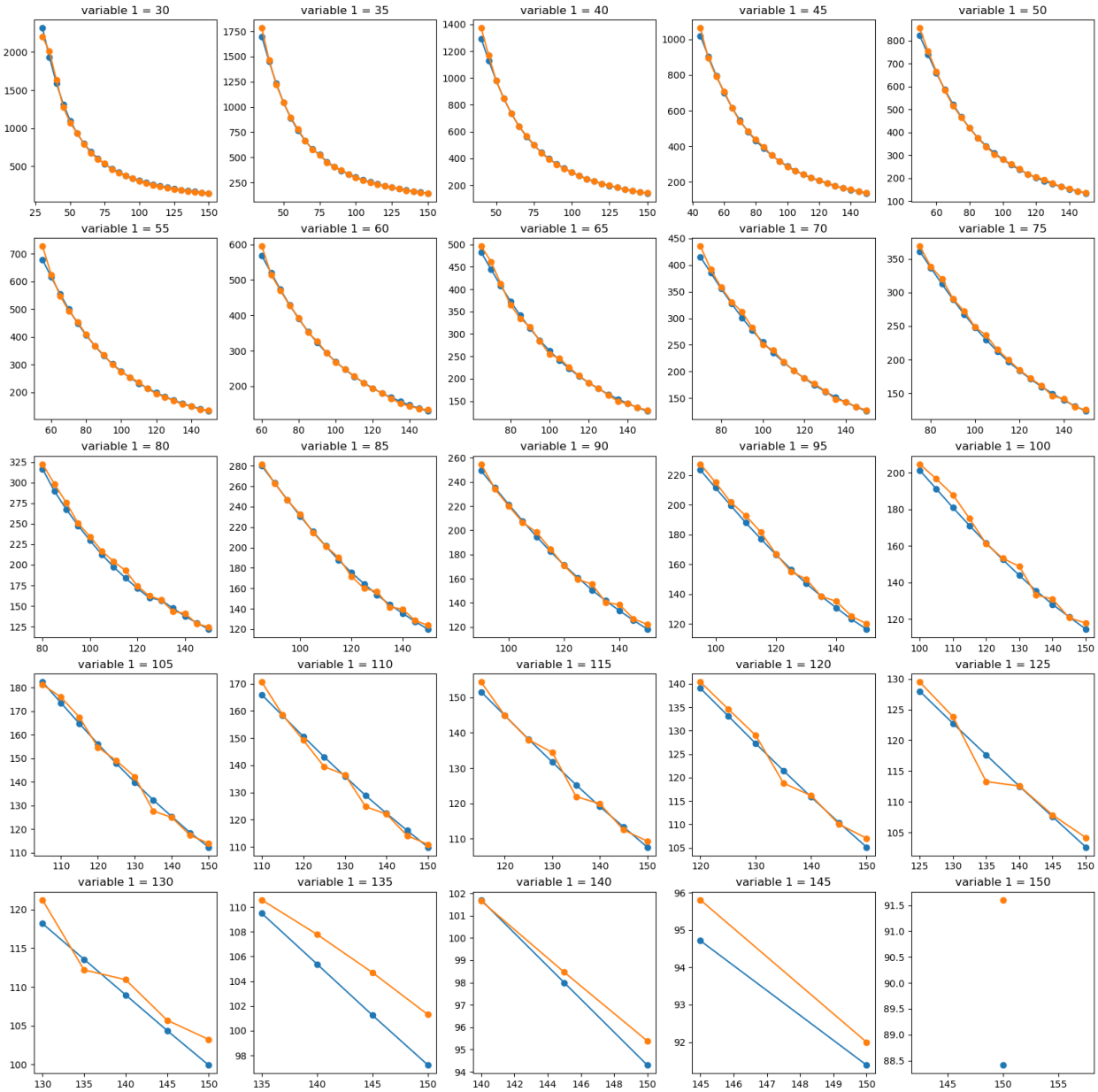
```
155.83846325125603
 0.9704475533876075
```

```
c:\Users\rajee\miniconda3\envs\mlenv\lib\site-packages\sklearn\
ensemble\_forest.py:611: UserWarning: Some inputs do not have OOB
scores. This probably means too few trees were used to compute any
reliable OOB estimates.
  warn(
```

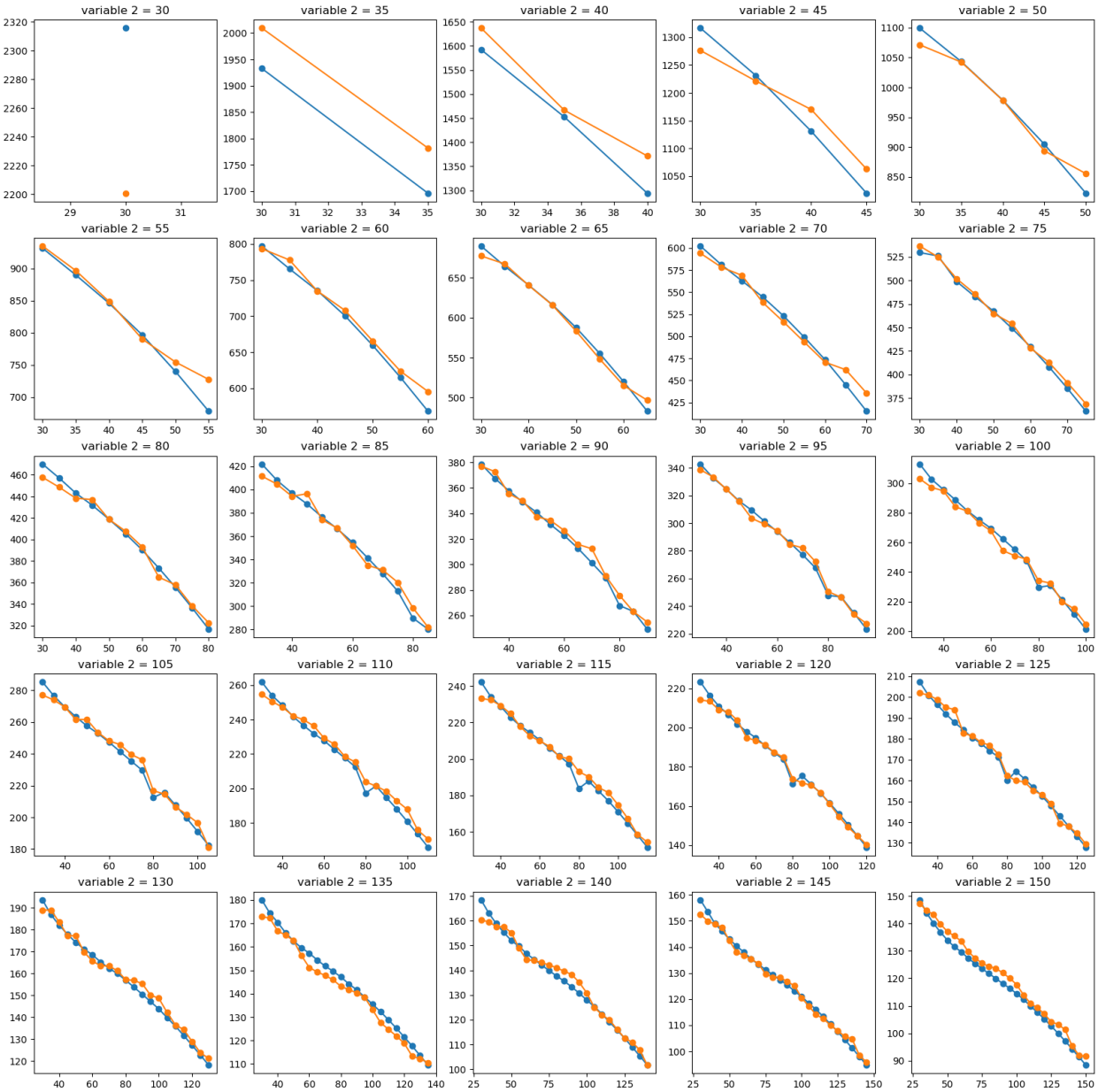
```
[estimator.get_depth() for estimator in RFReg.estimators_]
```

```
[8, 8, 8, 8, 8, 8, 8, 8, 8, 8]
```

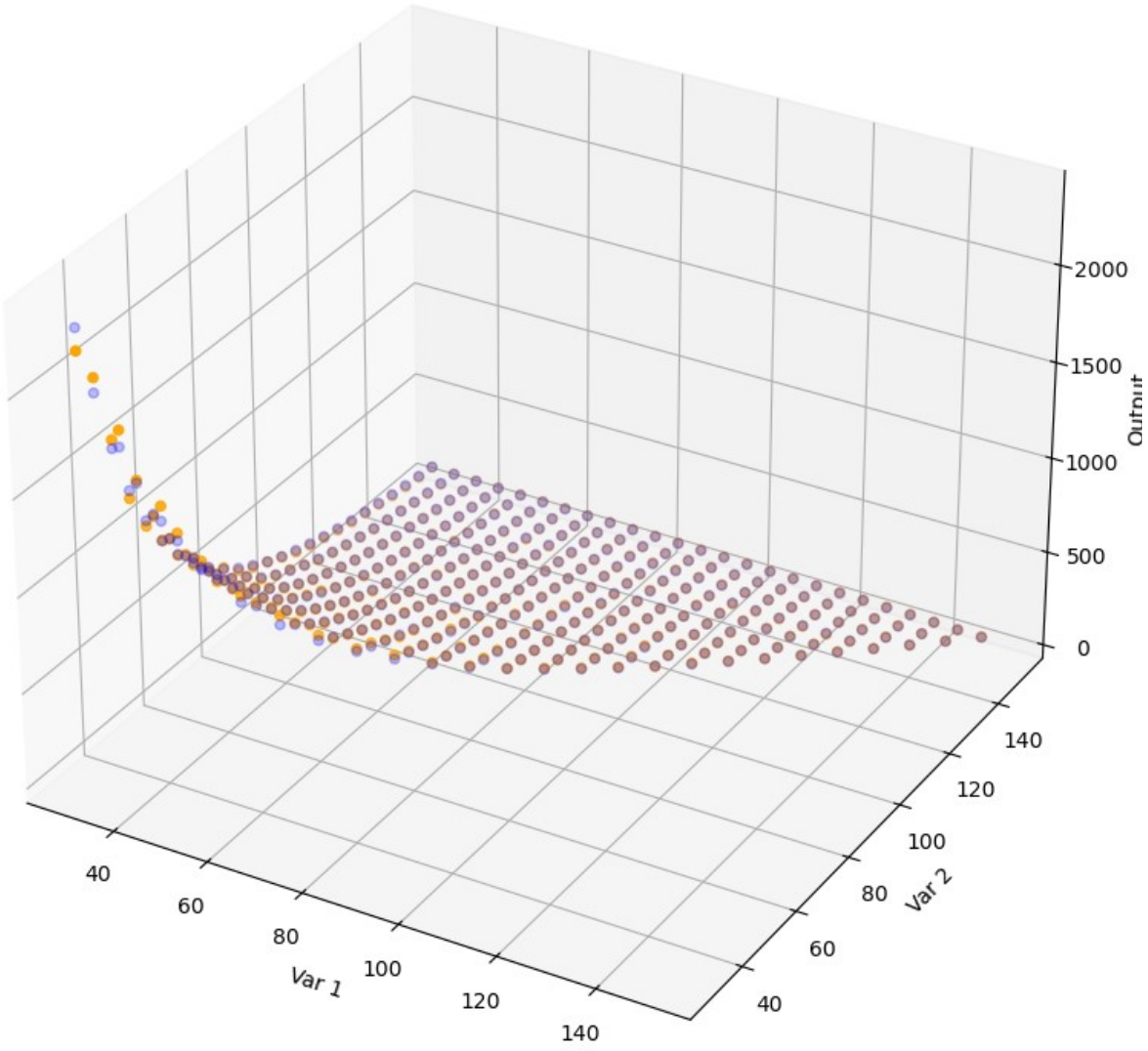
```
plot2d_var1const('RFReg')
```




```
plot2d_var2const('RFReg')
```




```
plot3d('RFReg')
```



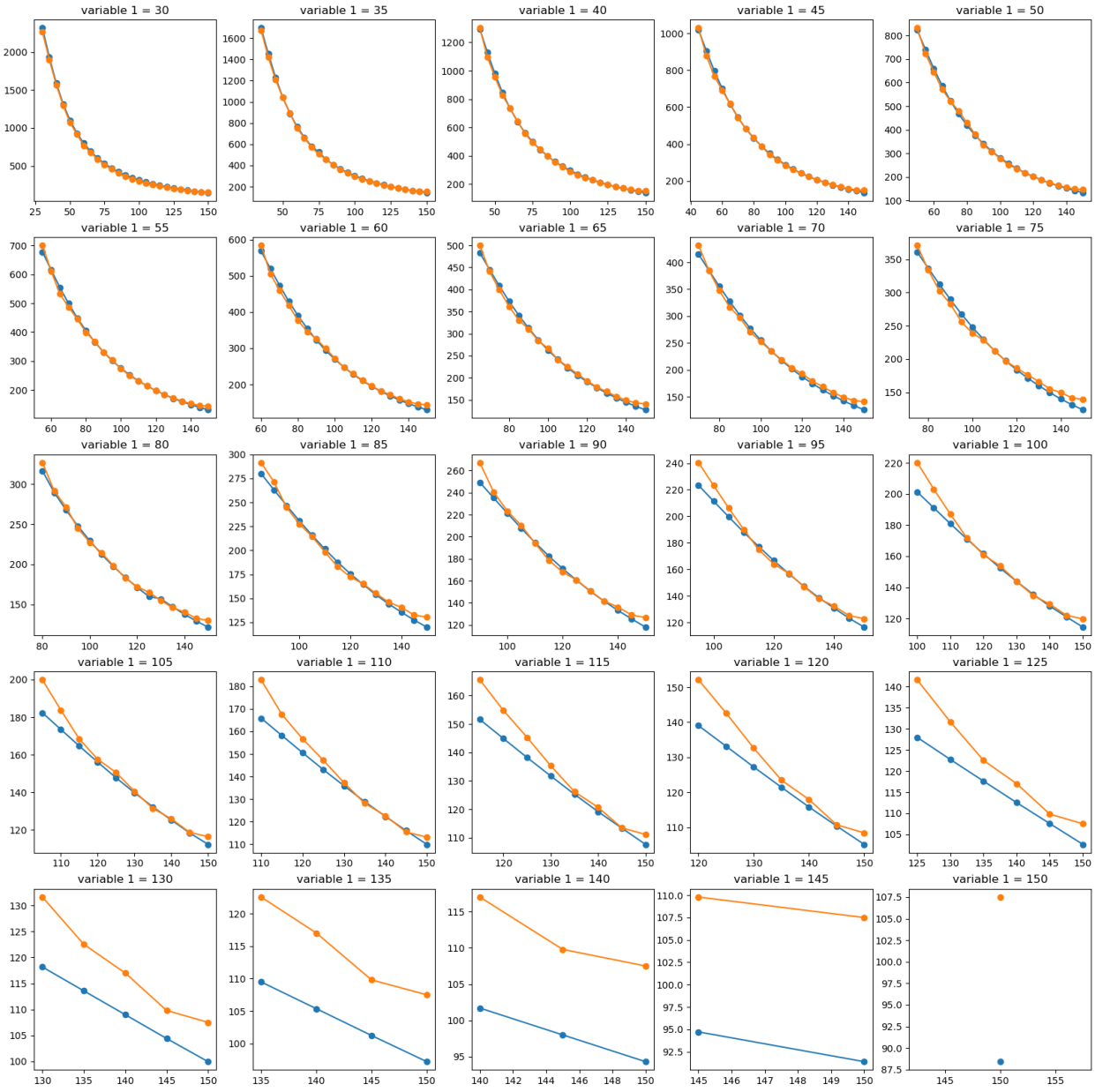
Gradient Boosting Regressor

```
GBR = GradientBoostingRegressor(loss = 'squared_error',
                                learning_rate = 0.1,
                                n_estimators = 40,
                                # max_depth = 3,
                                random_state = 0)

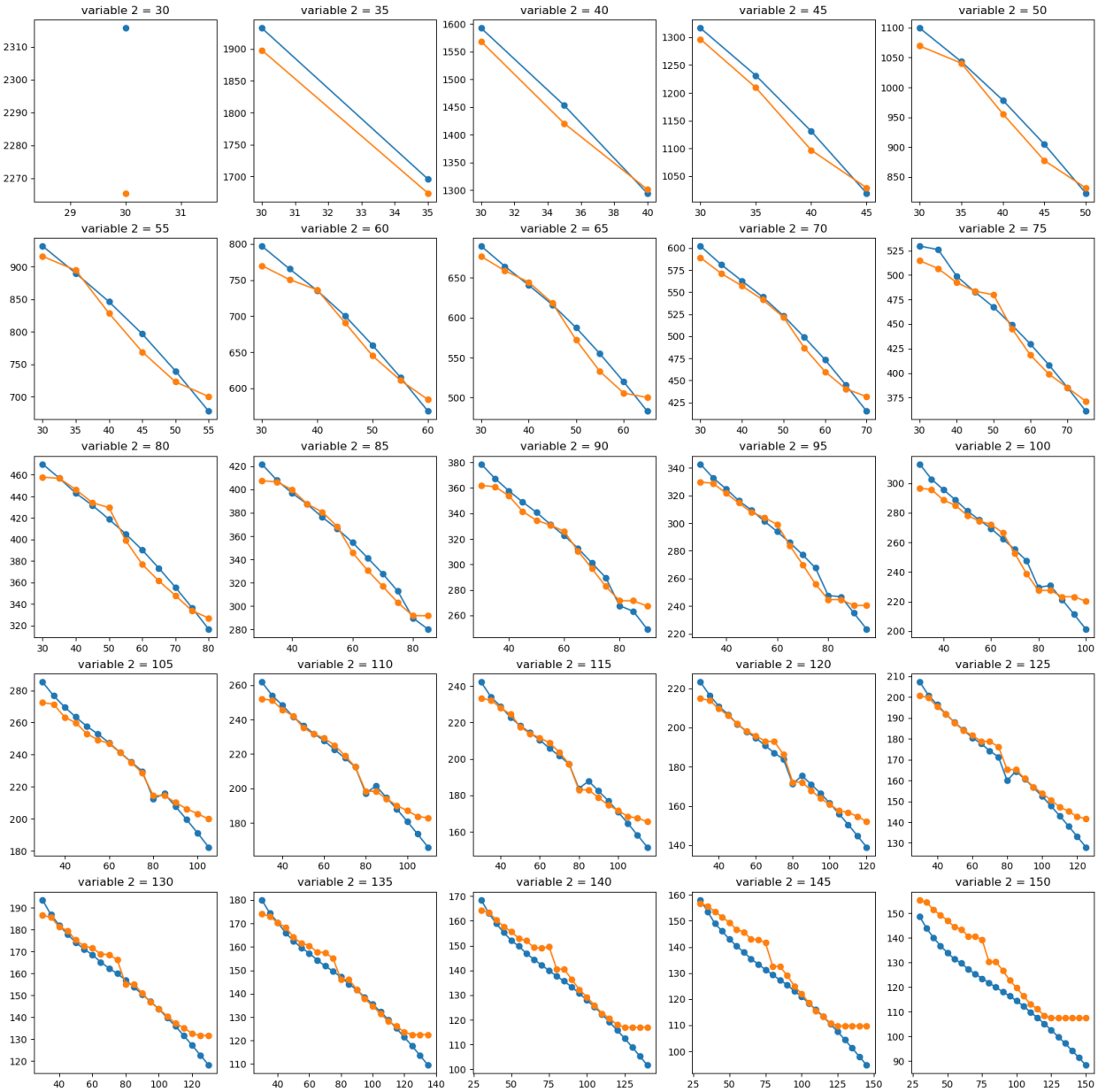
GBR.fit(X,Y)
df['GBR'] = GBR.predict(X)
print(mse(Y, df['GBR']))
```

```
92.55233764248817
```

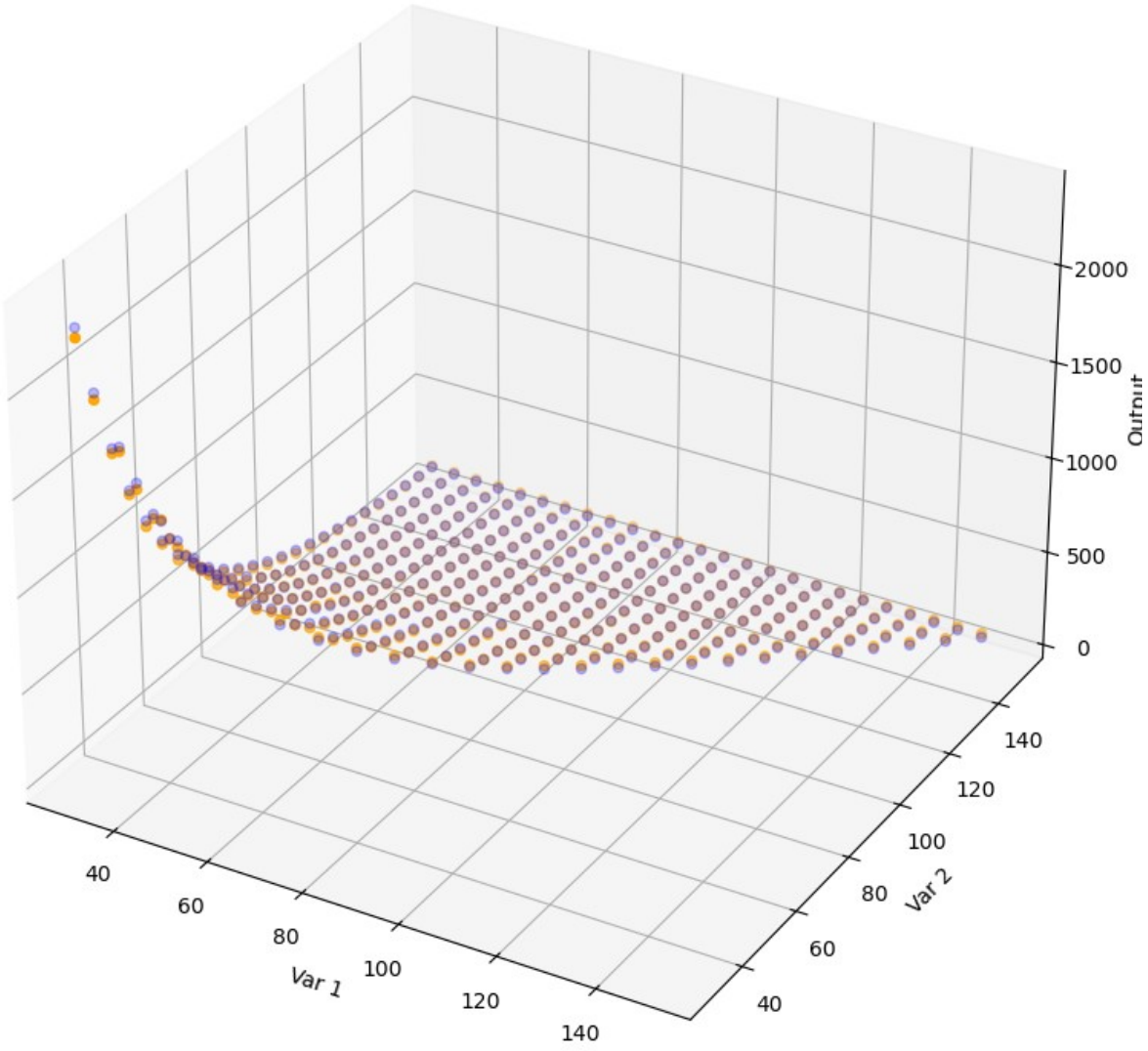
```
plot2d_var1const('GBR')
```




```
plot2d_var2const('GBR')
```


variable 2 = 30
variable 2 = 35
variable 2 = 40
variable 2 = 45
variable 2 = 50
variable 2 = 55
variable 2 = 60
variable 2 = 65
variable 2 = 70
variable 2 = 75
variable 2 = 80
variable 2 = 85
variable 2 = 90
variable 2 = 95
variable 2 = 100
variable 2 = 105
variable 2 = 110
variable 2 = 115
variable 2 = 120
variable 2 = 125
variable 2 = 130
variable 2 = 135
variable 2 = 140
variable 2 = 145
variable 2 = 150

```
plot3d('GBR')
```



AdaBoost Regressor

```
dtr = DecisionTreeRegressor(max_depth = 3)

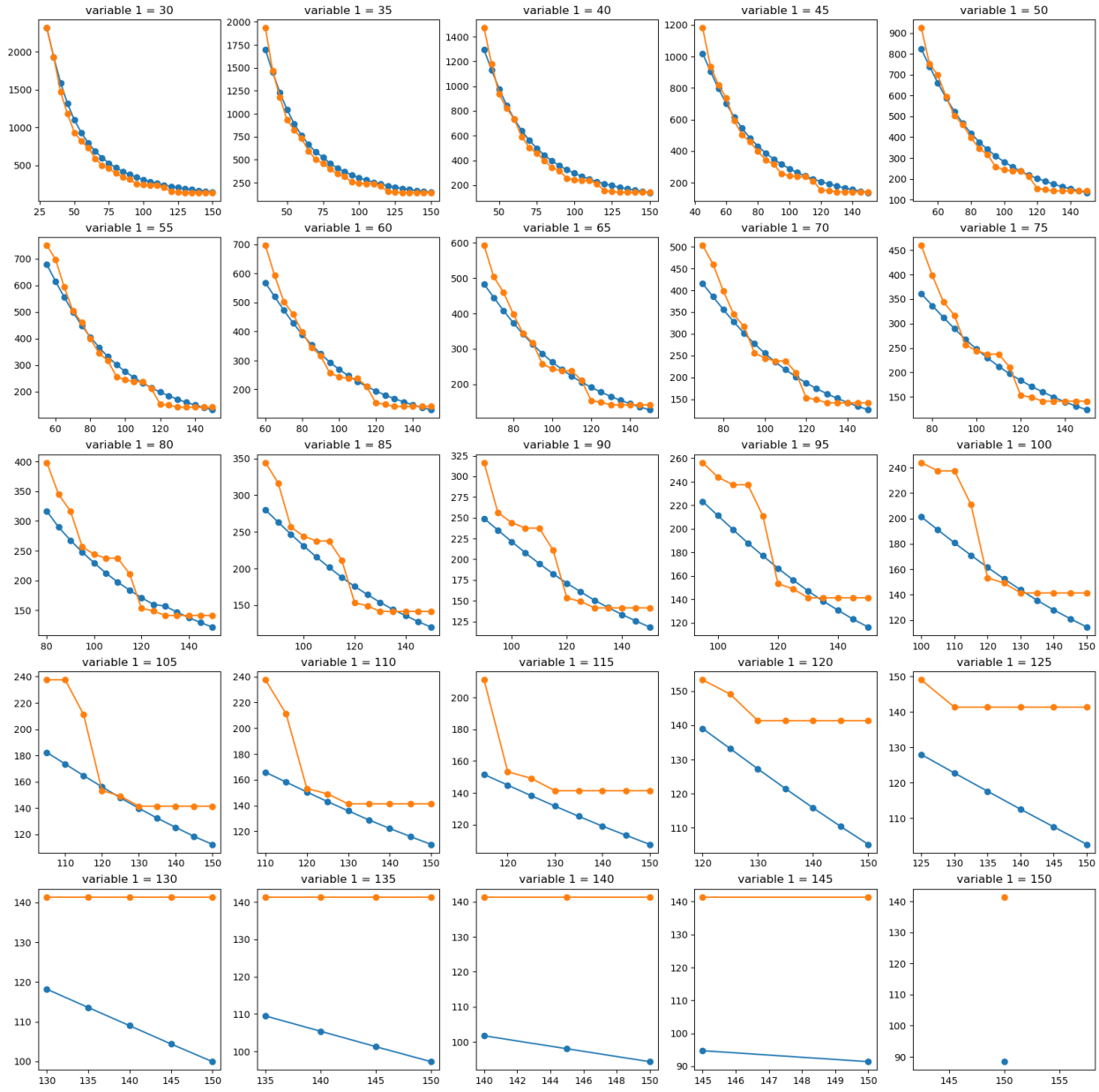
ABR = AdaBoostRegressor(estimator = dtr,
                        n_estimators = 10,
                        learning_rate = 0.1,
                        loss = 'linear',
                        random_state = 0)

ABR.fit(X,Y)
```

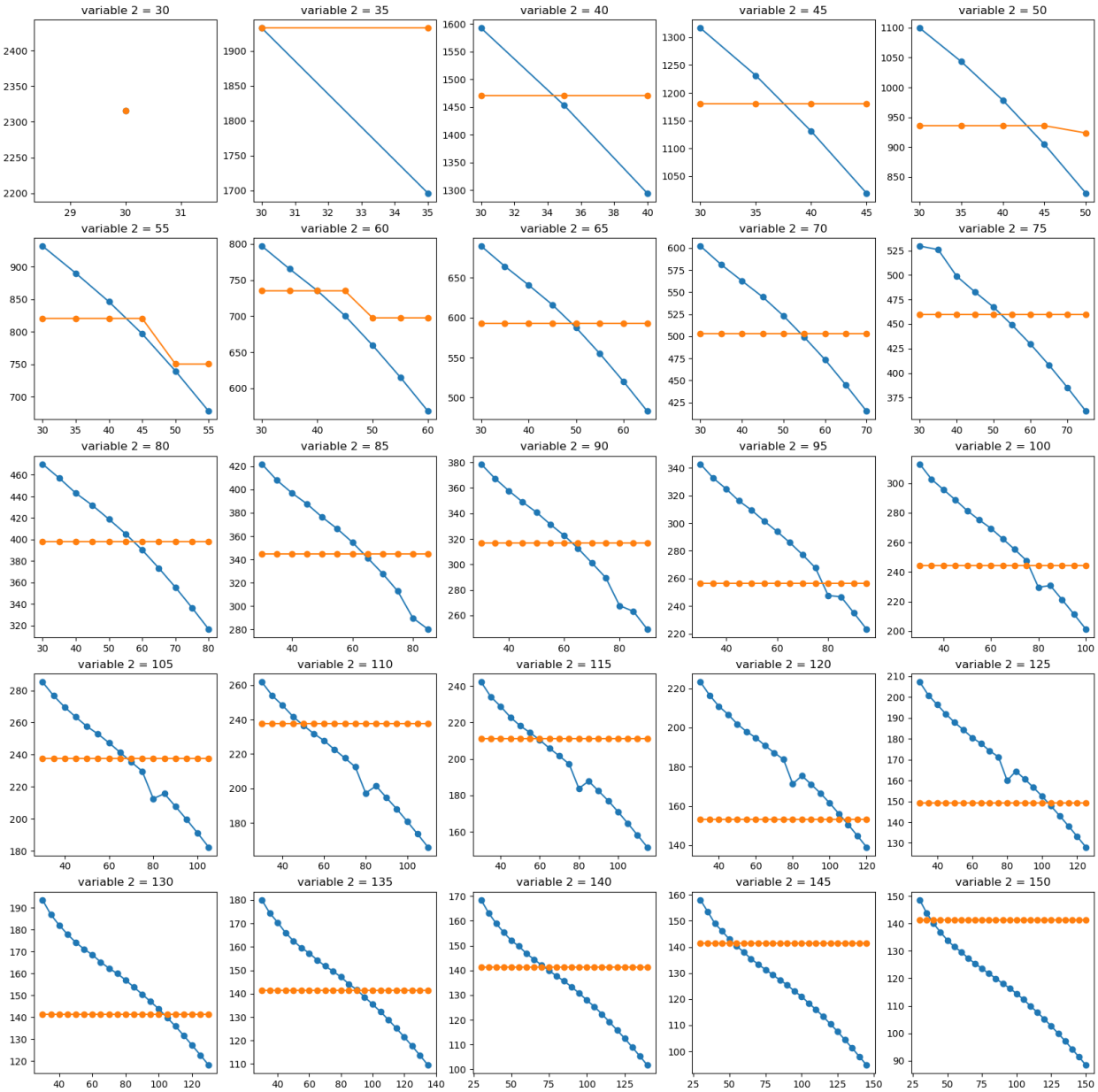
```
df['ABR'] = ABR.predict(X)
print(mse(Y, df['ABR']))
```

```
1970.873634056997
```

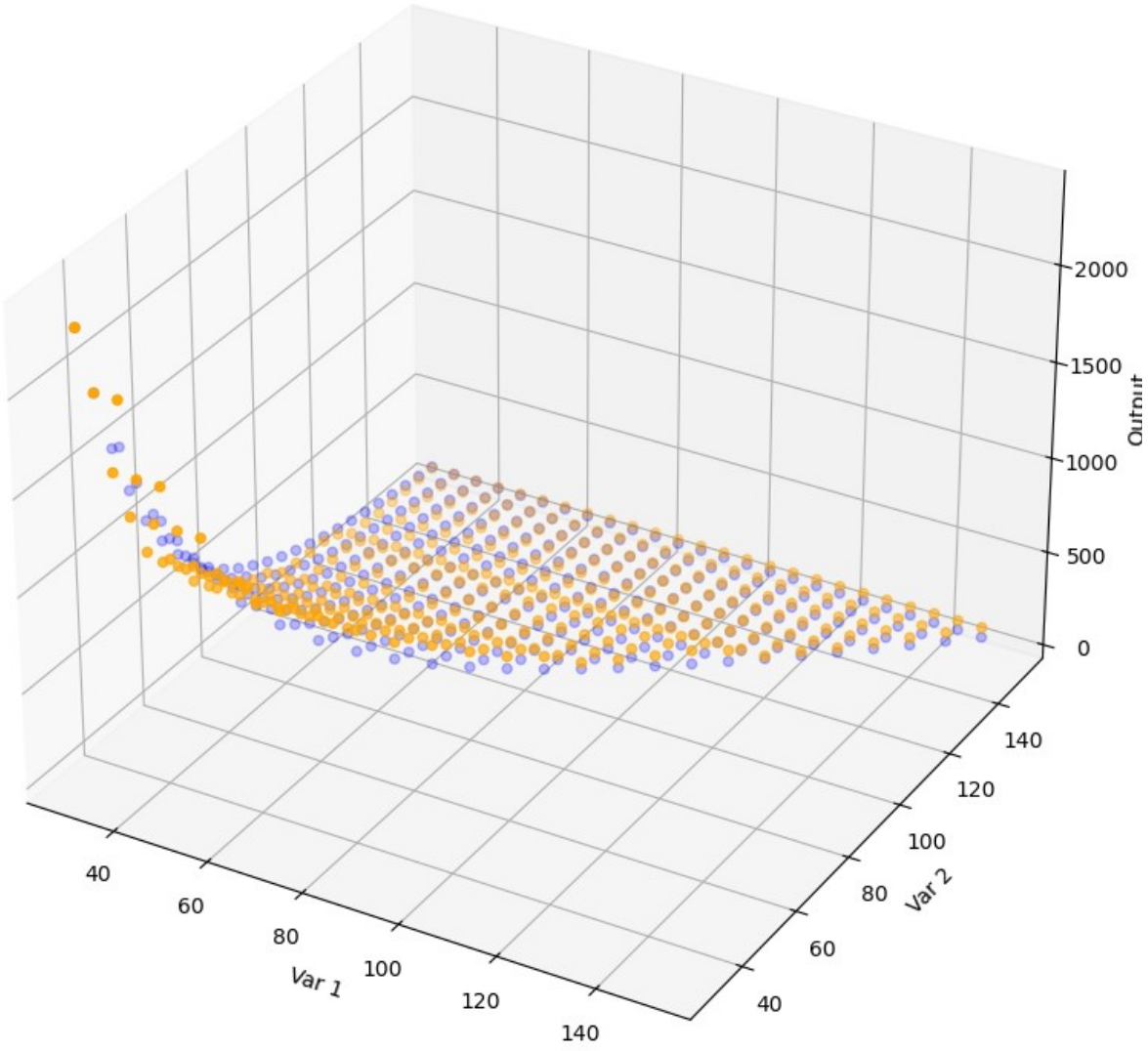
```
plot2d_var1const('ABR')
```




```
plot2d_var2const('ABR')
```


variable 2 = 30
variable 2 = 35
variable 2 = 40
variable 2 = 45
variable 2 = 50
variable 2 = 55
variable 2 = 60
variable 2 = 65
variable 2 = 70
variable 2 = 75
variable 2 = 80
variable 2 = 85
variable 2 = 90
variable 2 = 95
variable 2 = 100
variable 2 = 105
variable 2 = 110
variable 2 = 115
variable 2 = 120
variable 2 = 125
variable 2 = 130
variable 2 = 135
variable 2 = 140
variable 2 = 145
variable 2 = 150

```
plot3d('ABR')
```



XGBoost

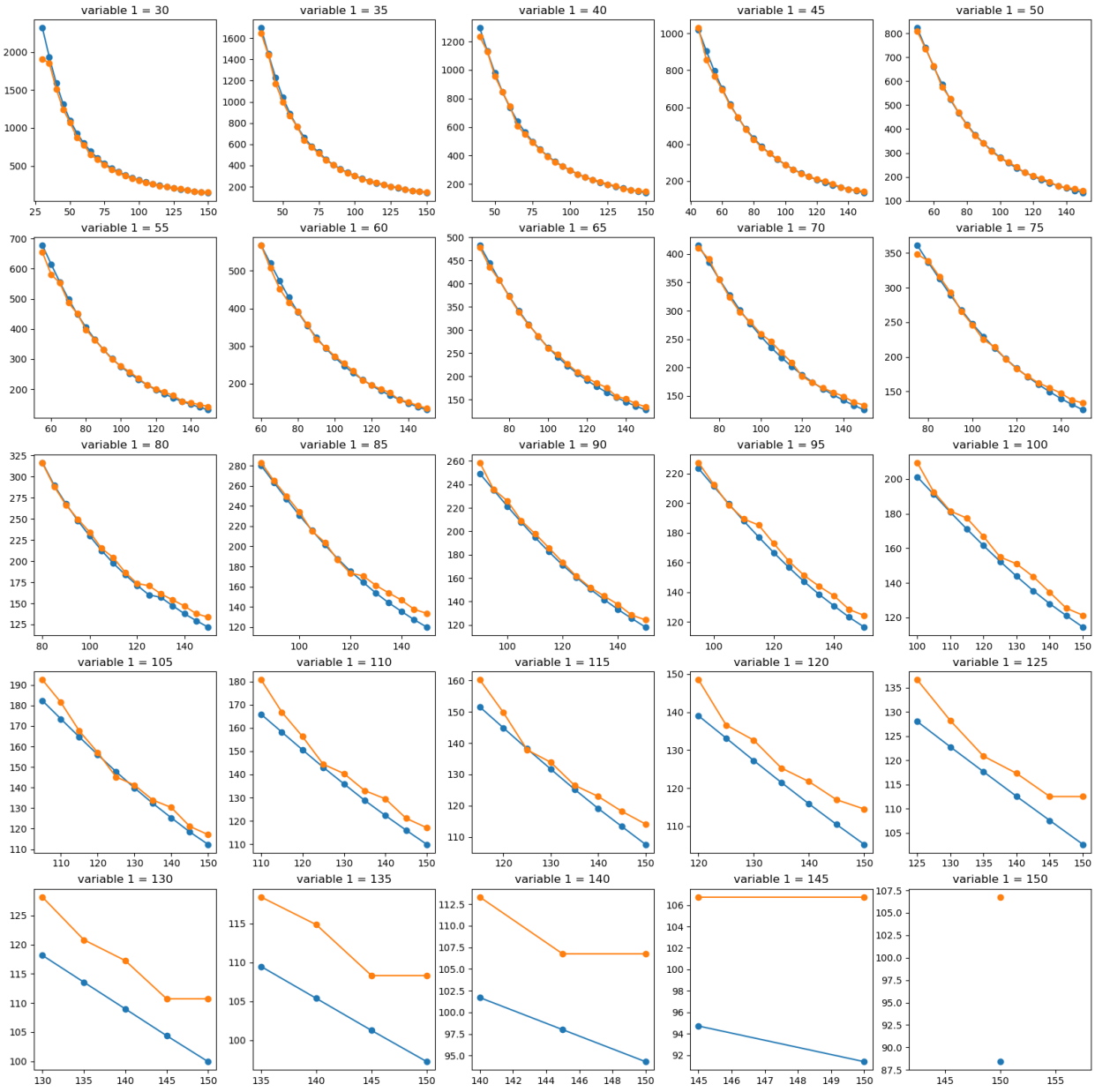
```
XGB = xgboost.XGBRegressor(objective = 'reg:squarederror',
                           n_estimators = 10,
                           random_state = 0)

XGB.fit(X,Y)
df['XGB'] = XGB.predict(X)

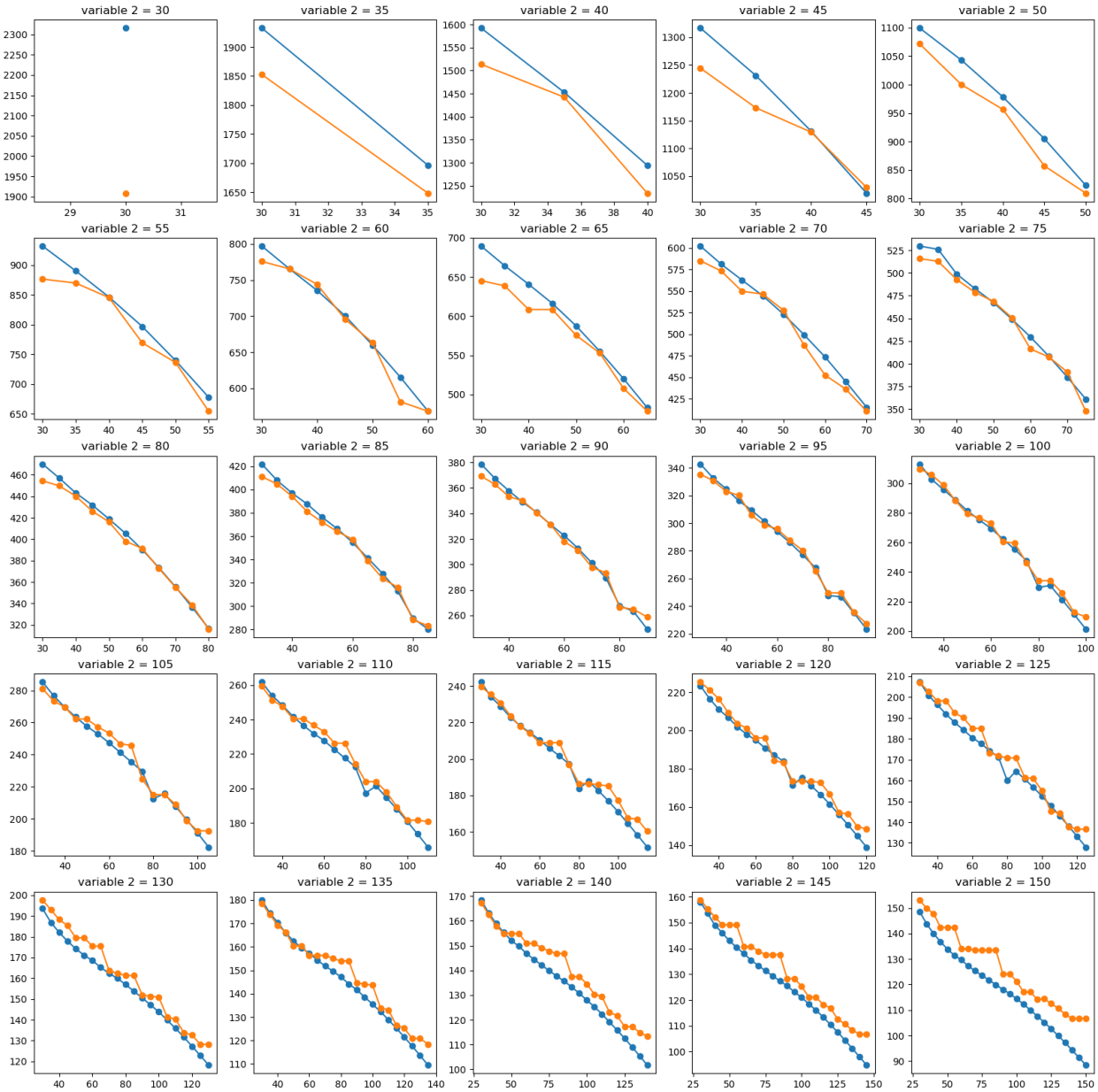
print(mse(Y, df['XGB']))
```

```
677.190049978358
```

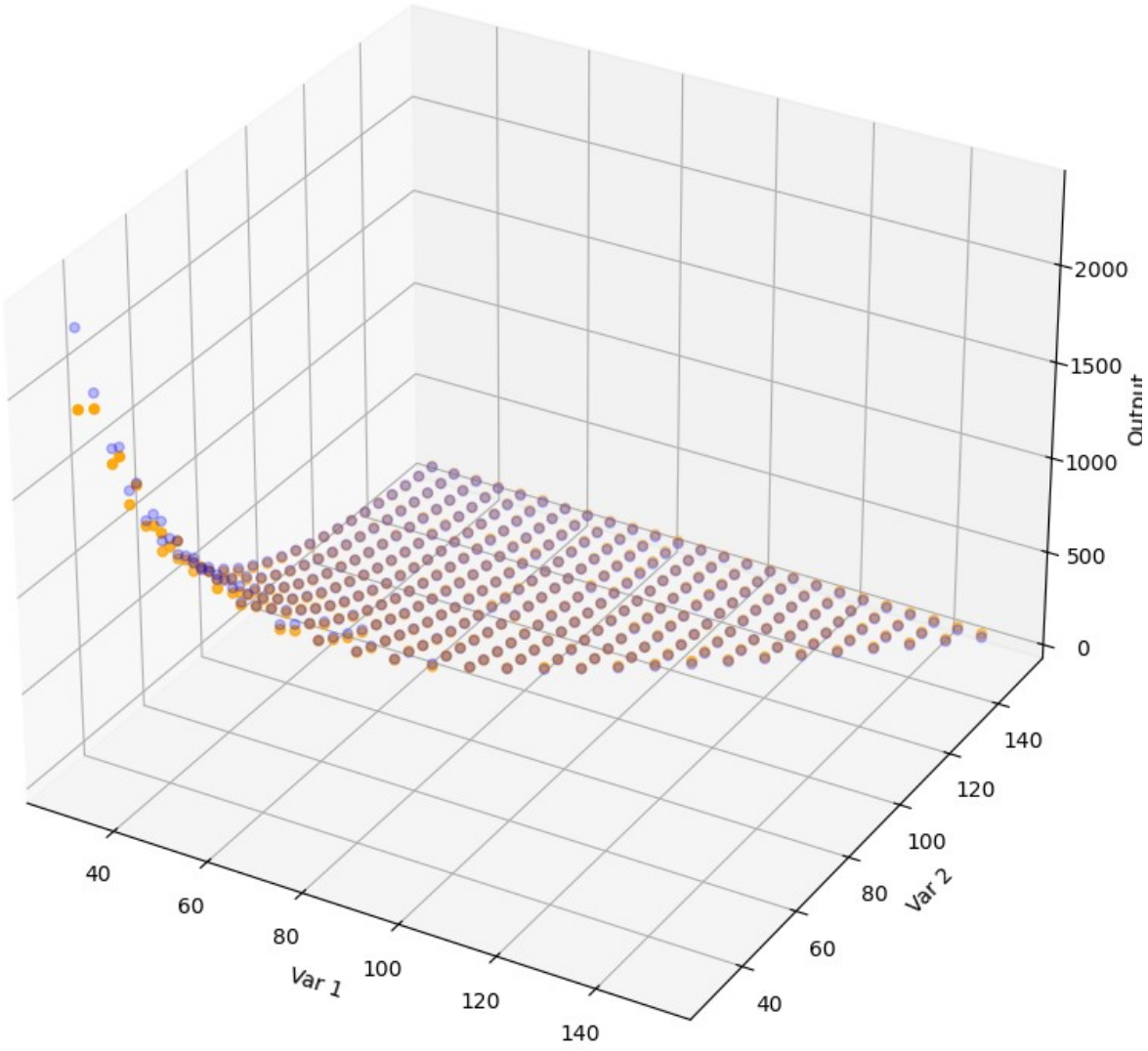
```
plot2d_var1const('XGB')
```




```
plot2d_var2const('XGB')
```


variable 2 = 30
variable 2 = 35
variable 2 = 40
variable 2 = 45
variable 2 = 50
variable 2 = 55
variable 2 = 60
variable 2 = 65
variable 2 = 70
variable 2 = 75
variable 2 = 80
variable 2 = 85
variable 2 = 90
variable 2 = 95
variable 2 = 100
variable 2 = 105
variable 2 = 110
variable 2 = 115
variable 2 = 120
variable 2 = 125
variable 2 = 130
variable 2 = 135
variable 2 = 140
variable 2 = 145
variable 2 = 150

```
plot3d('XGB')
```



LightGBM

```
LGBM = LGBMRegressor(metric = 'rmse')
LGBM.fit(X,Y)
df['LGBM'] = LGBM.predict(X)
print(mse(Y,df['LGBM']))
```

```
[LightGBM] [Warning] Found whitespace in feature_names, replace with
underlines
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead
of testing was 0.000038 seconds.
```

```
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 50
[LightGBM] [Info] Number of data points in the train set: 325, number
of used features: 2
[LightGBM] [Info] Start training from score 299.897862
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
```

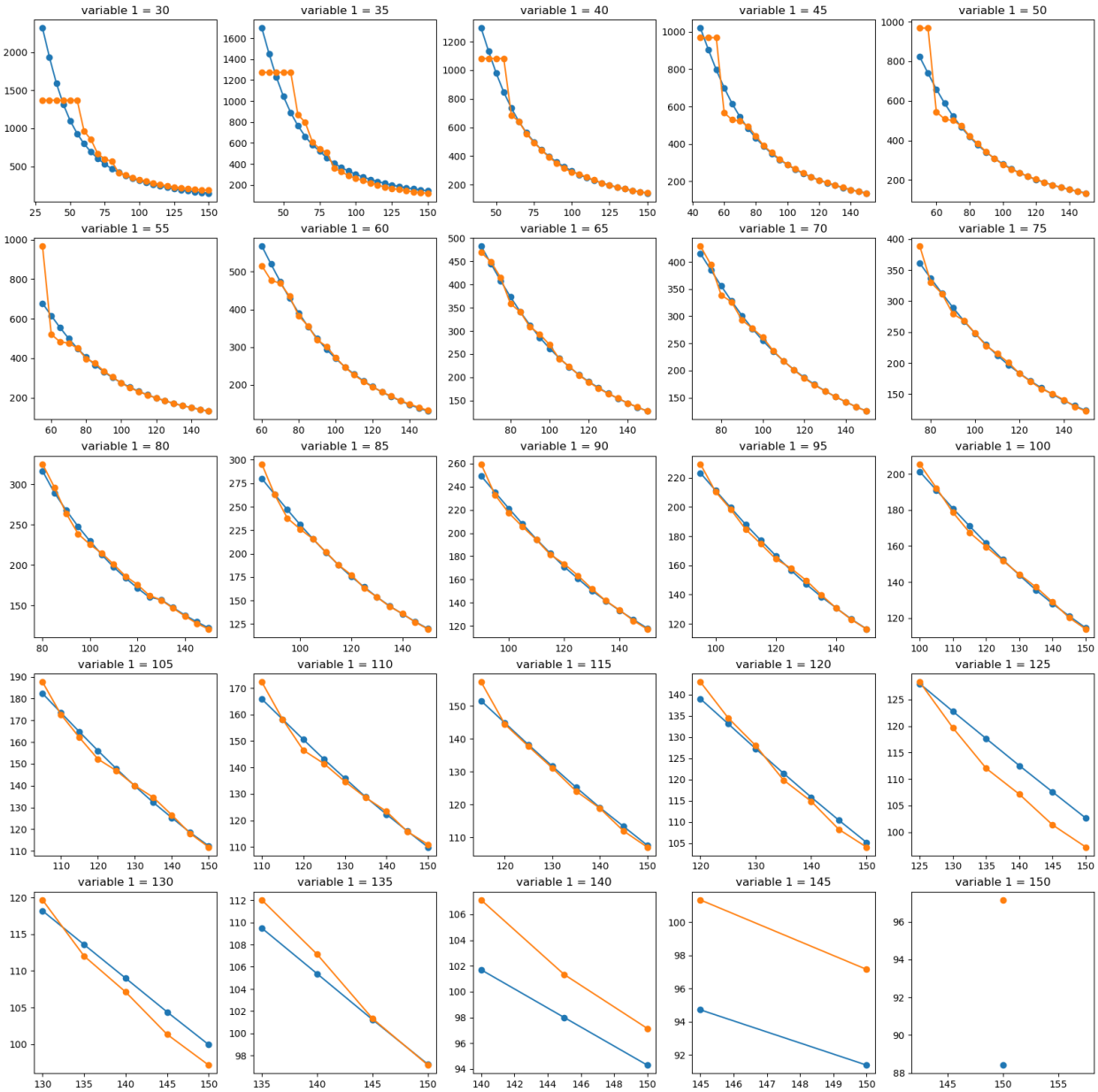
```
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
```

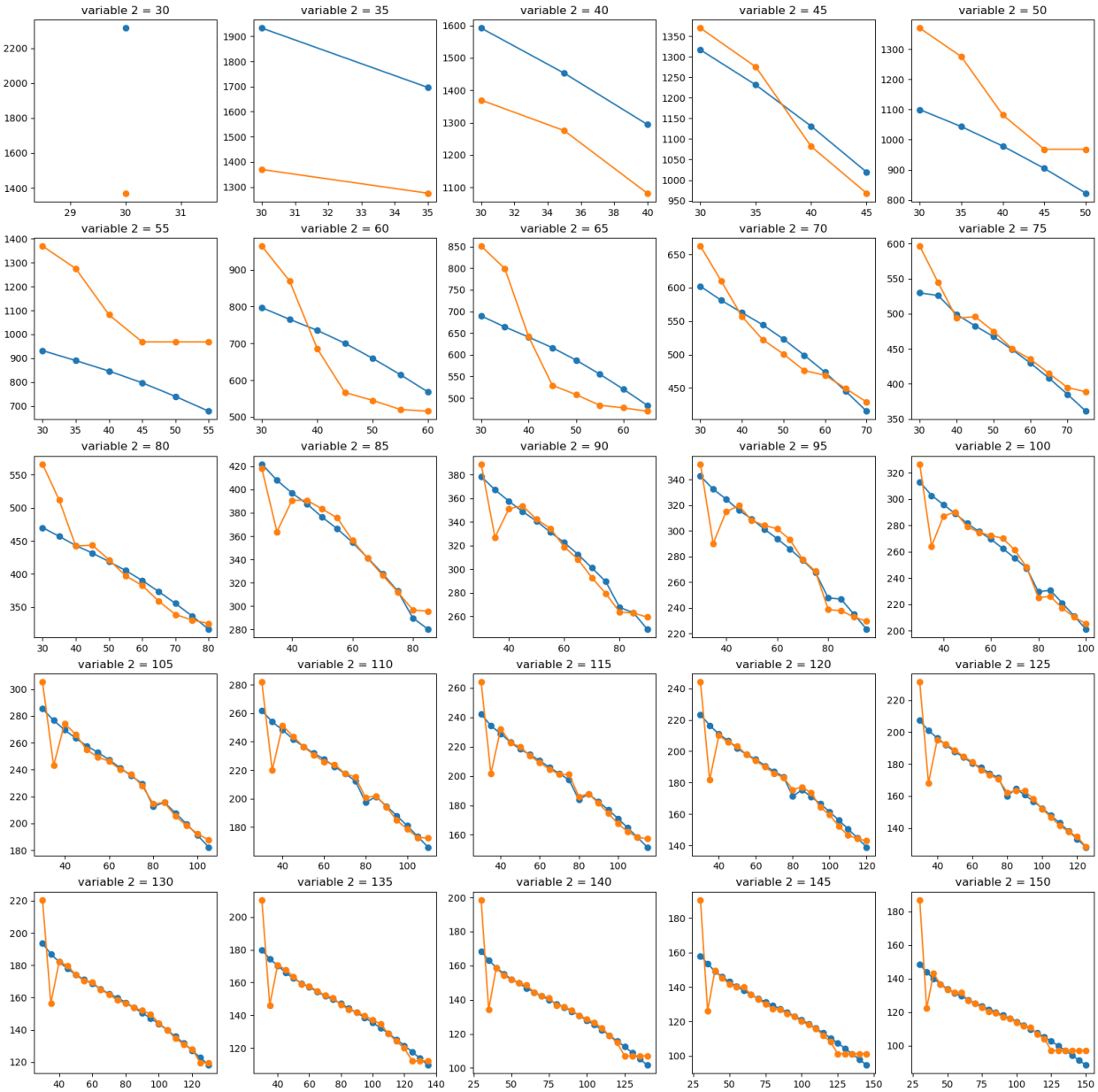
```
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
[LightGBM] [Warning] No further splits with positive gain, best gain:
-inf
7540.100271888015
```

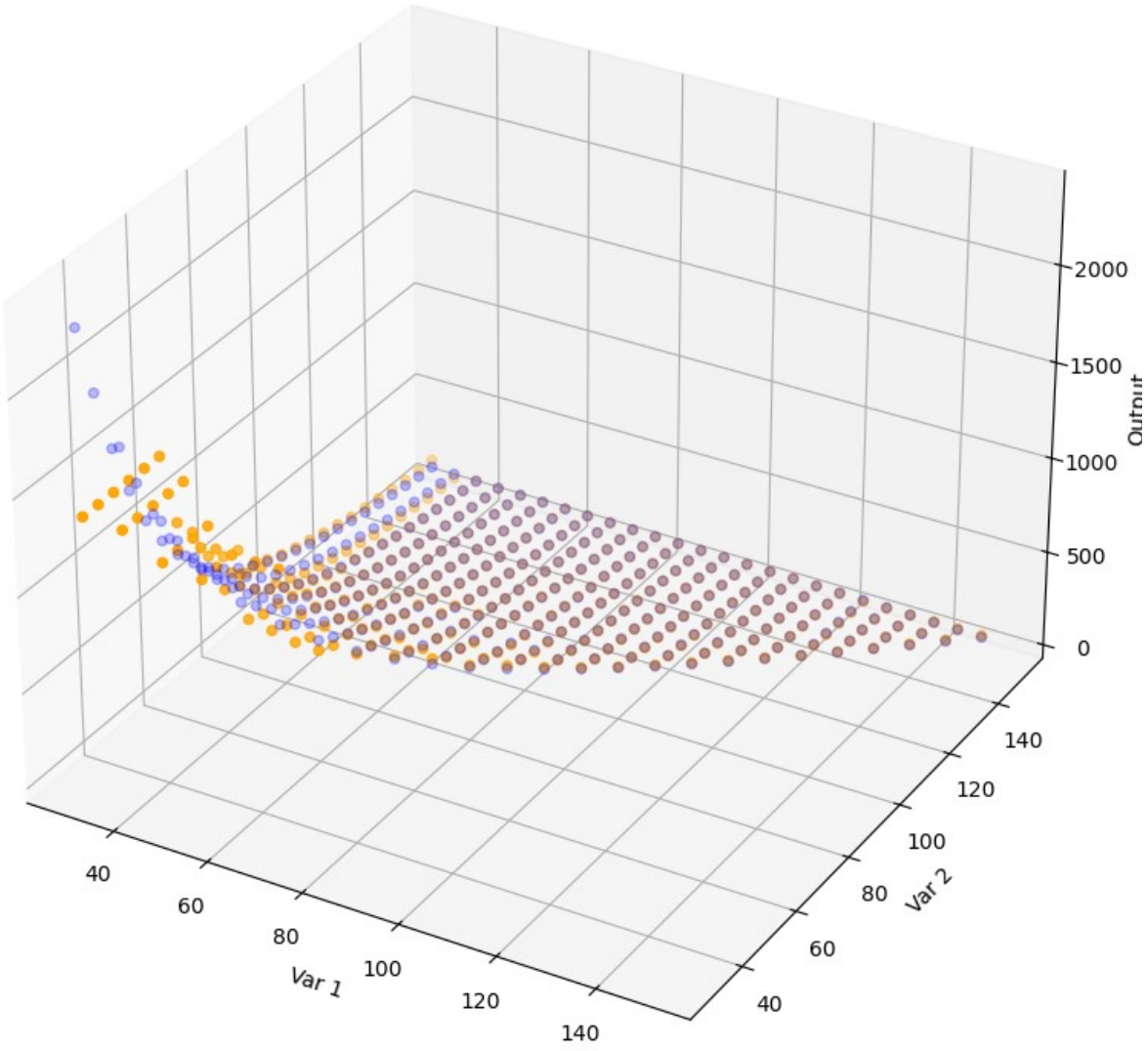
```
plot2d_var1const('LGBM')
```




```
plot2d_var2const('LGBM')
```


variable 2 = 30
variable 2 = 35
variable 2 = 40
variable 2 = 45
variable 2 = 50
variable 2 = 55
variable 2 = 60
variable 2 = 65
variable 2 = 70
variable 2 = 75
variable 2 = 80
variable 2 = 85
variable 2 = 90
variable 2 = 95
variable 2 = 100
variable 2 = 105
variable 2 = 110
variable 2 = 115
variable 2 = 120
variable 2 = 125
variable 2 = 130
variable 2 = 135
variable 2 = 140
variable 2 = 145
variable 2 = 150

```
plot3d('LGBM')
```



NN

```
scaler = StandardScaler()
X = scaler.fit_transform(X)

scaler_Y = StandardScaler()
Y = scaler_Y.fit_transform(Y.values.reshape(-1,1))

NN = tf.keras.models.Sequential([
    tf.keras.layers.Dense(10, activation = 'relu', input_shape =
(2,)),
```

```
    tf.keras.layers.Dense(10, activation = 'relu'),
    tf.keras.layers.Dense(1)
])

NN.compile(
    optimizer = 'adam',
    loss = 'mse',
    # metrics = ['mae']
)

history = NN.fit(X,Y, epochs = 5)

df['NN'] = scaler_Y.inverse_transform(NN.predict(X).reshape(-1,1))
print(mse(Y, df['NN']))
```

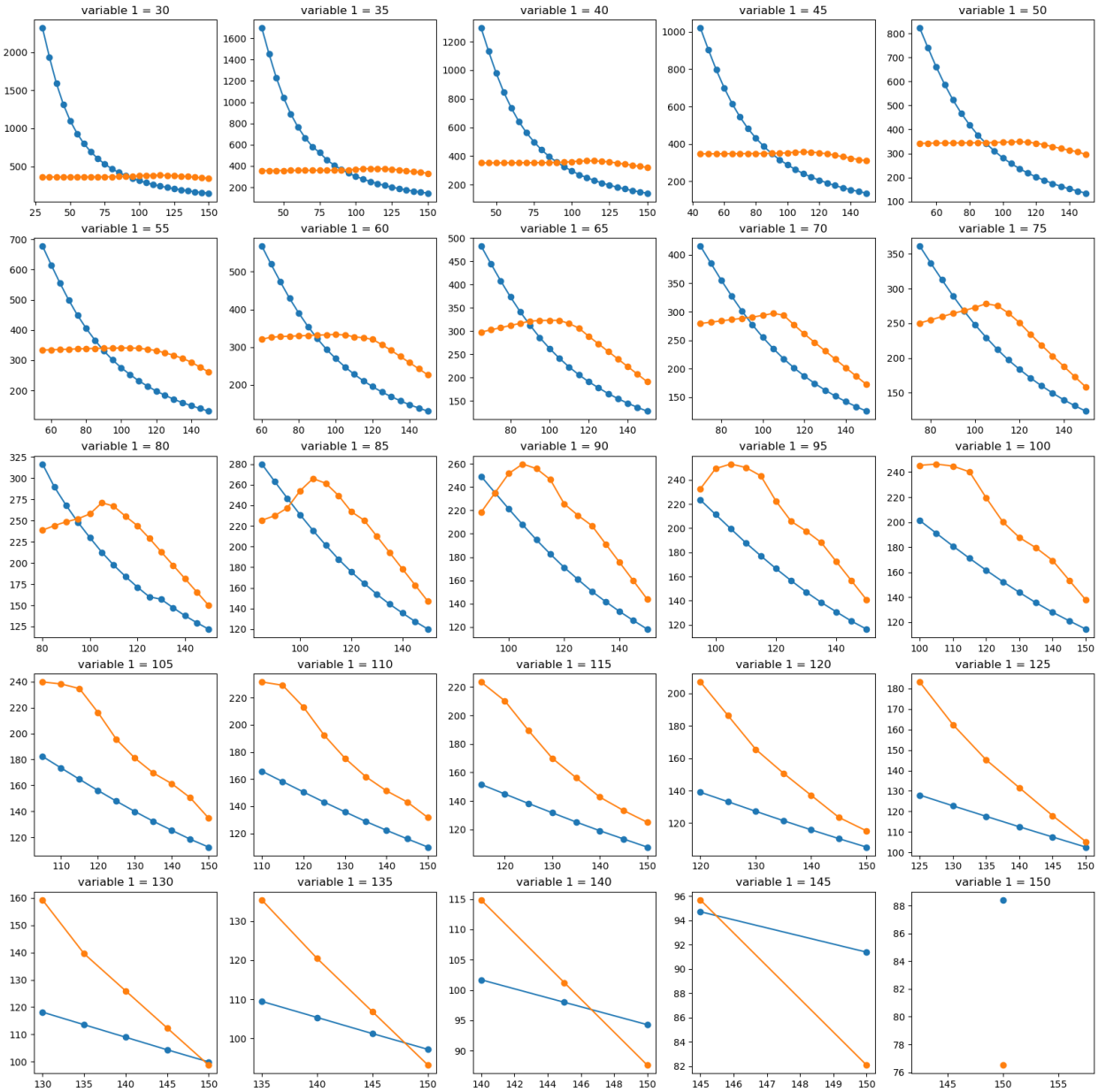
```
Epoch 1/5
11/11 [==============================] - 1s 2ms/step - loss: 1.1231
Epoch 2/5
11/11 [==============================] - 0s 2ms/step - loss: 1.0482
Epoch 3/5
11/11 [==============================] - 0s 2ms/step - loss: 0.9737
Epoch 4/5
11/11 [==============================] - 0s 2ms/step - loss: 0.9091
Epoch 5/5
11/11 [==============================] - 0s 2ms/step - loss: 0.8469
11/11 [==============================] - 0s 1ms/step
81148.77594270669
```

```
NN.evaluate(X,Y)
```

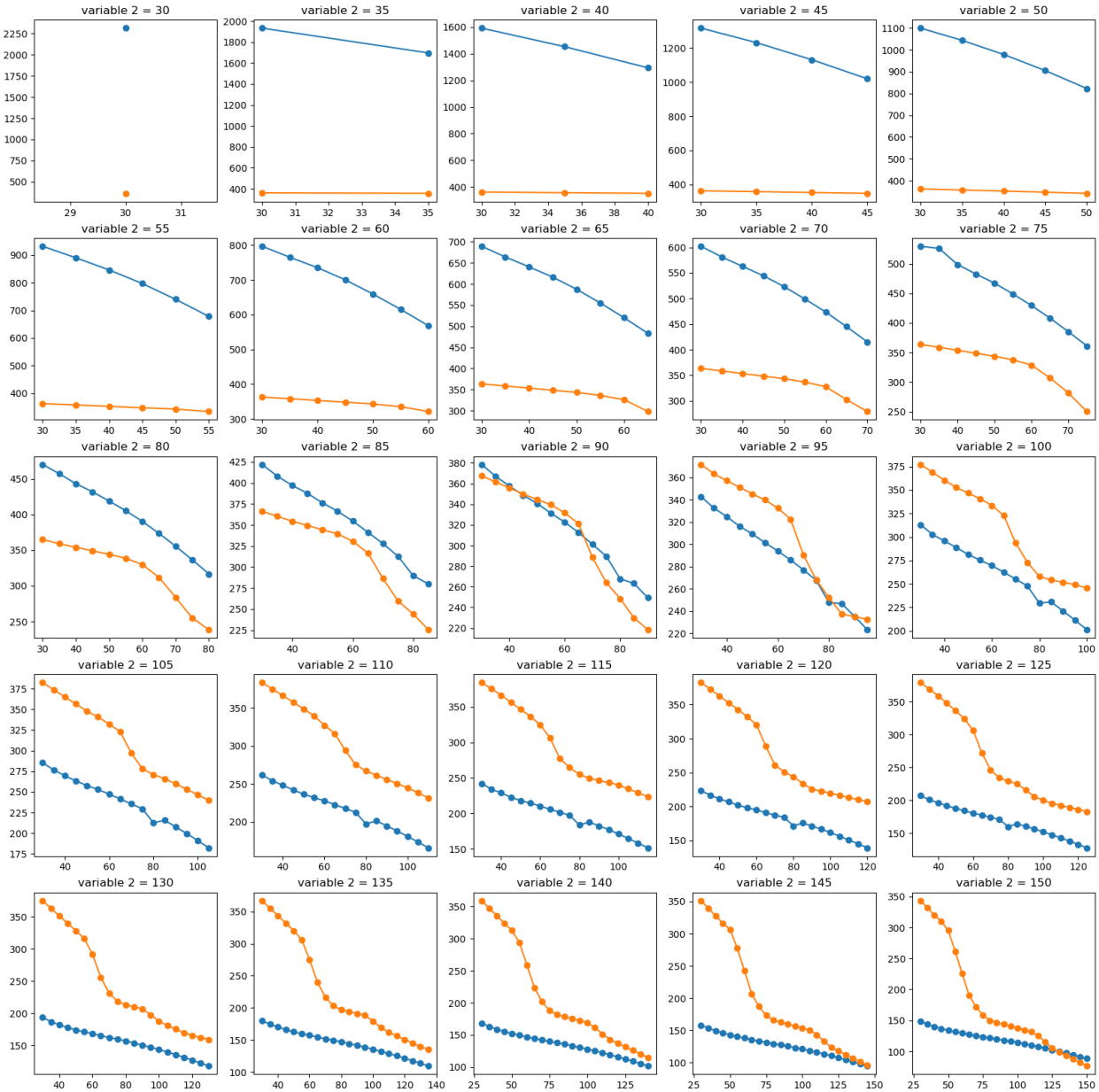
```
11/11 [==============================] - 0s 1ms/step - loss: 0.8061

0.8060632348060608
```

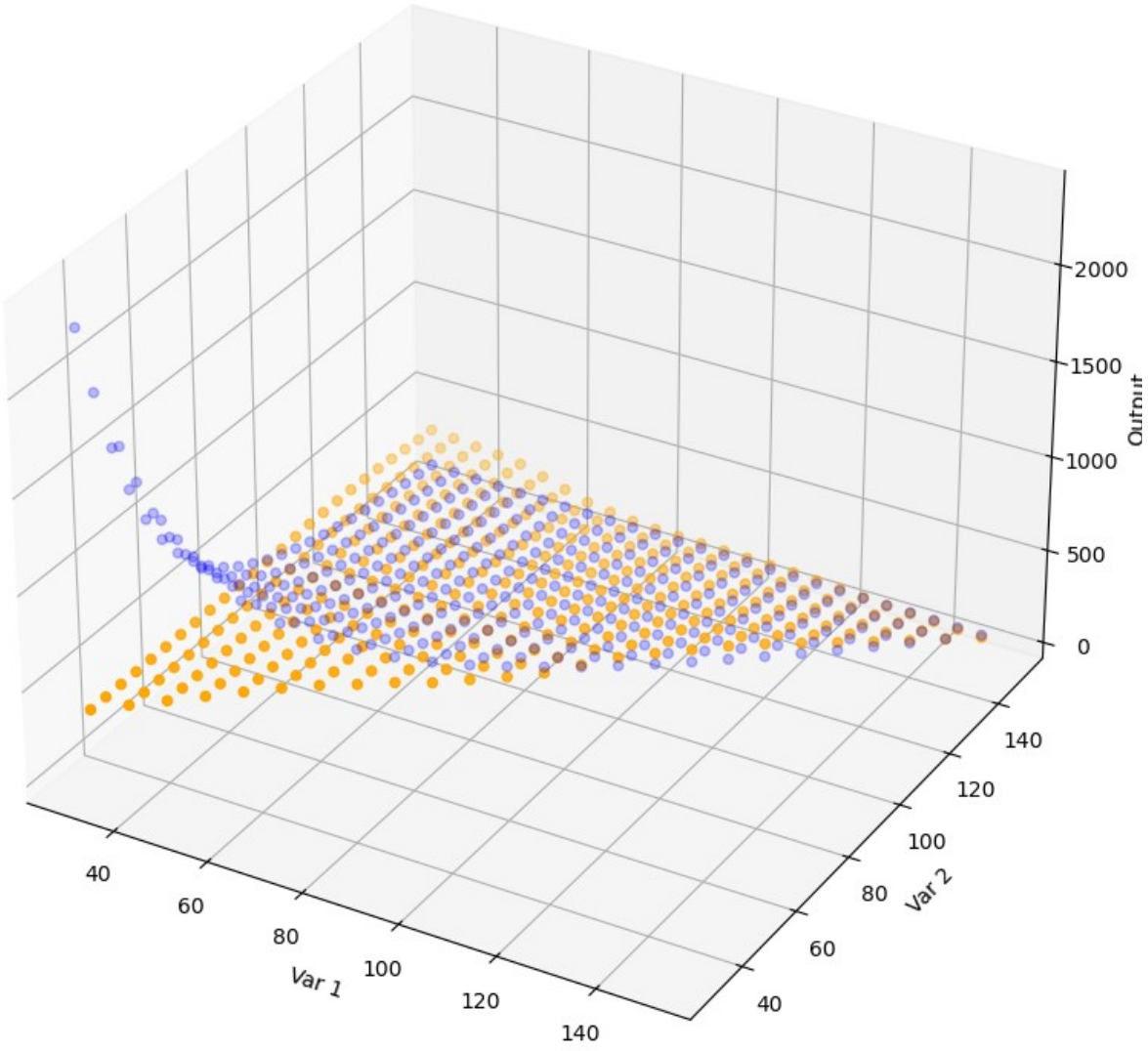
```
plot2d_var1const('NN')
```




```
plot2d_var2const('NN')
```


variable 2 = 30
variable 2 = 35
variable 2 = 40
variable 2 = 45
variable 2 = 50
variable 2 = 55
variable 2 = 60
variable 2 = 65
variable 2 = 70
variable 2 = 75
variable 2 = 80
variable 2 = 85
variable 2 = 90
variable 2 = 95
variable 2 = 100
variable 2 = 105
variable 2 = 110
variable 2 = 115
variable 2 = 120
variable 2 = 125
variable 2 = 130
variable 2 = 135
variable 2 = 140
variable 2 = 145
variable 2 = 150

```
plot3d('NN')
```




```
df['Output'].values
```

```
array([2315.7  , 1932.6  , 1592.2  , 1316.6  , 1099.5  ,  931.63 ,
        797.07 ,  689.49 ,  602.12 ,  529.58 ,  470.11 ,  421.96 ,
        378.61 ,  342.78 ,  313.01 ,  285.26 ,  261.88 ,  242.24 ,
        223.3  ,  207.23 ,  193.55 ,  180.01 ,  168.44 ,  158.01 ,
        148.54 , 1696.1  , 1452.8  , 1231.3  , 1043.3  ,  889.97 ,
        765.4  ,  664.18 ,  581.15 ,  525.95 ,  457.11 ,  407.98 ,
        367.27 ,  332.6  ,  302.65 ,  276.7  ,  253.97 ,  234.07 ,
        216.5  ,  200.86 ,  186.89 ,  174.43 ,  163.17 ,  153.58 ,
        143.8  , 1294.3  , 1131.   ,  978.57 ,  846.07 ,  735.4  ,
        640.69 ,  562.72 ,  498.81 ,  443.08 ,  396.97 ,  357.7  ,
        324.93 ,  295.72 ,  269.6  ,  248.26 ,  228.84 ,  210.91 ,
```

```
       196.34 ,  182.08 ,  170.44 ,  158.89 ,  148.99 ,  139.98 ,
      1019.4  ,  904.73 ,  796.7  ,  700.53 ,  616.17 ,  544.54 ,
       482.61 ,  431.74 ,  387.57 ,  348.78 ,  316.23 ,  288.78 ,
       263.4  ,  241.83 ,  222.88 ,  206.75 ,  191.85 ,  177.9  ,
       166.   ,  155.29 ,  146.07 ,  136.79 ,  822.54 ,  739.72 ,
       659.51 ,  587.49 ,  523.12 ,  467.27 ,  418.82 ,  376.2  ,
       340.8  ,  309.41 ,  281.47 ,  257.61 ,  236.62 ,  218.17 ,
       201.82 ,  187.78 ,  174.2  ,  162.53 ,  152.02 ,  142.98 ,
       133.89 ,  677.9  ,  615.29 ,  554.99 ,  499.02 ,  448.95 ,
       405.27 ,  366.22 ,  331.31 ,  301.45 ,  275.26 ,  252.85 ,
       231.88 ,  214.47 ,  197.98 ,  184.25 ,  171.1  ,  159.57 ,
       149.73 ,  140.36 ,  131.48 ,  568.15 ,  520.05 ,  473.58 ,
       429.49 ,  390.26 ,  354.64 ,  322.61 ,  293.95 ,  269.51 ,
       247.4  ,  227.76 ,  210.39 ,  194.85 ,  180.52 ,  168.54 ,
       157.29 ,  146.79 ,  138.03 ,  129.68 ,  482.97 ,  444.99 ,
       408.16 ,  373.4  ,  341.33 ,  312.54 ,  285.93 ,  262.38 ,
       241.41 ,  222.58 ,  205.81 ,  190.82 ,  177.76 ,  165.21 ,
       154.36 ,  144.43 ,  135.45 ,  127.31 ,  415.28 ,  385.25 ,
       355.56 ,  327.73 ,  301.14 ,  277.11 ,  255.25 ,  235.48 ,
       217.64 ,  201.65 ,  187.16 ,  174.18 ,  162.39 ,  151.79 ,
       142.12 ,  133.35 ,  125.37 ,  361.04 ,  336.54 ,  312.65 ,
       289.6  ,  267.69 ,  247.69 ,  229.62 ,  212.52 ,  197.27 ,
       183.81 ,  171.3  ,  159.94 ,  149.58 ,  139.89 ,  131.34 ,
       123.53 ,  316.69 ,  289.6  ,  267.69 ,  247.69 ,  229.62 ,
       212.52 ,  197.27 ,  183.81 ,  171.3  ,  159.94 ,  157.   ,
       147.1  ,  137.65 ,  129.35 ,  121.74 ,  280.01 ,  263.19 ,
       246.79 ,  230.8  ,  215.67 ,  201.4  ,  187.8  ,  175.47 ,
       164.43 ,  153.74 ,  144.15 ,  135.63 ,  127.34 ,  119.94 ,
       249.26 ,  235.18 ,  221.24 ,  207.7  ,  194.69 ,  182.49 ,
       171.04 ,  160.68 ,  150.51 ,  141.63 ,  133.24 ,  125.48 ,
       118.08 ,  223.36 ,  211.39 ,  199.63 ,  187.97 ,  177.1  ,
       166.55 ,  156.63 ,  147.17 ,  138.53 ,  130.72 ,  123.08 ,
       116.41 ,  201.3  ,  191.08 ,  180.84 ,  171.04 ,  161.52 ,
       152.39 ,  143.78 ,  135.47 ,  127.91 ,  121.04 ,  114.4  ,
       182.32 ,  173.46 ,  164.68 ,  156.07 ,  147.77 ,  139.82 ,
       132.28 ,  125.29 ,  118.42 ,  112.32 ,  165.86 ,  158.2  ,
       150.56 ,  143.06 ,  135.92 ,  128.85 ,  122.32 ,  116.01 ,
       109.9  ,  151.58 ,  144.86 ,  138.17 ,  131.67 ,  125.24 ,
       119.1  ,  113.33 ,  107.6  ,  139.02 ,  133.15 ,  127.24 ,
       121.47 ,  115.84 ,  110.41 ,  105.17 ,  127.99 ,  122.75 ,
       117.62 ,  112.52 ,  107.55 ,  102.61 ,  118.17 ,  113.57 ,
       108.99 ,  104.39 ,   99.937,  109.48 ,  105.37 ,  101.24 ,
        97.219,  101.7  ,   97.999,   94.303,   94.712,   91.383,
        88.412])

df['NN'].values

array([361.195   , 361.48834 , 361.7817  , 362.07504 , 362.36838 ,
       362.66174 , 362.95508 , 363.24844 , 363.54175 , 363.59116 ,
       364.89236 , 366.27673 , 367.66107 , 371.76782 , 377.18    ,
```

```
      382.3328  , 383.28232 , 384.2318  , 382.52728 , 378.91995 ,
      374.7821  , 366.89932 , 359.01654 , 351.13373 , 343.25095 ,
      356.4613  , 356.7546  , 357.04797 , 357.3413  , 357.63464 ,
      357.928   , 358.22134 , 358.51468 , 358.808   , 358.9669  ,
      360.35123 , 361.7356  , 363.3586  , 368.77078 , 373.69214 ,
      374.64163 , 375.59116 , 372.72217 , 368.88803 , 363.1179  ,
      355.2351  , 347.3523  , 339.4695  , 331.58673 , 351.72757 ,
      352.0209  , 352.31424 , 352.60757 , 352.9009  , 353.19427 ,
      353.4876  , 353.78094 , 354.07428 , 354.42575 , 355.8101  ,
      357.19446 , 360.3616  , 365.0515  , 366.001   , 366.49988 ,
      362.69028 , 358.85614 , 351.45364 , 343.57086 , 335.68808 ,
      327.8053  , 320.00684 , 346.99384 , 347.28714 , 347.5805  ,
      347.87384 , 348.16718 , 348.4605  , 348.75388 , 349.0472  ,
      349.34055 , 349.8846  , 351.26898 , 352.65332 , 356.41086 ,
      357.36038 , 356.4925  , 352.65836 , 347.6722  , 339.78943 ,
      331.90665 , 324.0253  , 316.4062  , 309.98907 , 341.9056  ,
      342.55344 , 342.84677 , 343.1401  , 343.43344 , 343.72678 ,
      344.02014 , 344.31348 , 344.6068  , 345.34348 , 346.72784 ,
      347.7702  , 348.54193 , 346.46057 , 342.62643 , 336.008   ,
      328.12518 , 320.31616 , 313.03156 , 306.61447 , 295.11554 ,
      333.90805 , 334.8536  , 335.7992  , 336.74478 , 337.69037 ,
      338.63596 , 339.28638 , 339.5797  , 339.87308 , 340.80234 ,
      340.69547 , 339.5707  , 336.42865 , 332.22656 , 324.34375 ,
      316.607   , 306.12997 , 293.90695 , 277.54214 , 261.17728 ,
      321.02606 , 325.88785 , 327.80164 , 328.74722 , 329.69278 ,
      330.63837 , 331.58395 , 332.52954 , 333.47513 , 332.1013  ,
      327.09756 , 324.60406 , 320.39026 , 306.39886 , 291.34854 ,
      274.9837  , 258.6189  , 242.25406 , 225.88924 , 297.726   ,
      302.48557 , 307.16626 , 311.84695 , 316.52765 , 321.20837 ,
      322.57867 , 322.8912  , 322.8322  , 316.05536 , 306.2263  ,
      288.79013 , 272.4253  , 256.0605  , 239.69565 , 223.33081 ,
      206.96599 , 190.60117 , 279.31546 , 281.61063 , 283.9058  ,
      286.201   , 288.49615 , 290.54623 , 293.62024 , 296.966   ,
      294.04648 , 277.17337 , 261.00656 , 246.14261 , 231.32565 ,
      216.50871 , 201.69176 , 186.87482 , 172.05785 , 250.24895 ,
      254.8888  , 259.5287  , 264.09937 , 268.09875 , 272.8538  ,
      278.0452  , 275.22714 , 264.75818 , 250.90688 , 234.27145 ,
      218.45842 , 203.03662 , 188.08522 , 173.13382 , 158.1824  ,
      238.54167 , 244.03453 , 248.55031 , 252.0774  , 257.8986  ,
      271.03864 , 266.91104 , 254.85411 , 243.83786 , 229.04605 ,
      213.01163 , 197.21545 , 181.41928 , 165.62312 , 149.83293 ,
      225.525   , 229.86075 , 237.48225 , 254.13522 , 265.87354 ,
      261.37488 , 249.50584 , 233.81097 , 225.56984 , 209.97954 ,
      194.18391 , 178.38774 , 162.59157 , 146.79538 , 218.411   ,
      235.06398 , 251.71695 , 259.80853 , 255.83875 , 246.40846 ,
      225.61926 , 215.68454 , 206.94502 , 191.15167 , 175.35623 ,
      159.56004 , 143.76384 , 232.6457  , 249.29868 , 253.13197 ,
      250.3026  , 243.3111  , 222.5219  , 205.79922 , 197.5581  ,
      188.11717 , 172.3238  , 156.52849 , 140.73232 , 245.3969  ,
```

```
       246.45541 , 244.76645 , 240.21371 , 219.42453 , 200.20683 ,
       187.67279 , 179.43166 , 169.28929 , 153.49594 , 137.70078 ,
       239.77885 , 238.2675  , 234.73167 , 216.32716 , 195.53795 ,
       181.03288 , 169.54637 , 161.30522 , 150.46141 , 134.66808 ,
       231.59093 , 229.19554 , 213.22977 , 192.44058 , 175.46701 ,
       161.85889 , 151.41989 , 143.17877 , 131.63354 , 223.40303 ,
       210.1324  , 189.3432  , 169.90115 , 156.29303 , 142.68492 ,
       133.29347 , 125.052345, 207.03503 , 186.24583 , 165.45663 ,
       150.72717 , 137.11906 , 123.51093 , 115.167046, 183.14845 ,
       162.35925 , 145.1613  , 131.55319 , 117.94509 , 105.2817  ,
       159.26189 , 139.59541 , 125.98733 , 112.37922 ,  98.771095,
       135.3753  , 120.421486, 106.81335 ,  93.20525 , 114.85563 ,
       101.24749 ,  87.6394  ,  95.68166 ,  82.073494,  76.50764 ],
      dtype=float32)
```

All Results

```
df
```

```
     variable 1  variable 2    Output       custom      Linreg
DTReg  \
0            30          30  2315.700  2275.557132  929.492625
2315.700000
1            30          35  1932.600  1878.507927  892.620457
1814.350000
2            30          40  1592.200  1561.149755  855.748288
1446.433333
3            30          45  1316.600  1306.986398  818.876120
1133.378333
4            30          50  1099.500  1102.936387  782.003951
1133.378333
..          ...         ...       ...          ...         ...
...
320         140         145    97.999    96.875945  -27.577436
122.609127
321         140         150    94.303    88.224427  -64.449604
122.609127
322         145         145    94.712    94.593612  -32.532444
122.609127
323         145         150    91.383    86.181977  -69.404612
122.609127
324         150         150    88.412    84.139528  -74.359621
122.609127

           RFReg          GBR          ABR          XGB
LGBM  \
0    2200.770000  2265.459092  2315.700000  1908.496094  1369.554931
```

```
1    2009.220000  1898.043524  1932.600000  1852.588379  1369.554931

2    1636.990000  1568.093904  1469.985714  1513.691650  1369.554931

3    1276.330000  1296.628279  1180.877778  1244.475830  1369.554931

4    1071.473000  1069.554477   935.830000  1071.644531  1369.554931

..           ...          ...         ...          ...          ...

320    98.464250   109.795820   141.344716   106.754135   101.333581

321    95.387582   107.505190   141.344716   106.754135    97.150664

322    95.804054   109.795820   141.344716   106.754135   101.333581

323    91.997586   107.505190   141.344716   106.754135    97.150664

324    91.603152   107.505190   141.344716   106.754135    97.150664


              NN
0    361.195007
1    361.488342
2    361.781708
3    362.075043
4    362.368378
..          ...
320  101.247490
321   87.639397
322   95.681664
323   82.073494
324   76.507637

[325 rows x 12 columns]
```

```
# df.to_csv('All models results.csv')
```